

DASC7606 -7

Deep Learning

Natural Language Processing Embedding, Transformer – BERT

Dr. Bethany Chan
Professor Francis Chin

2025

NLP (2) – Outline

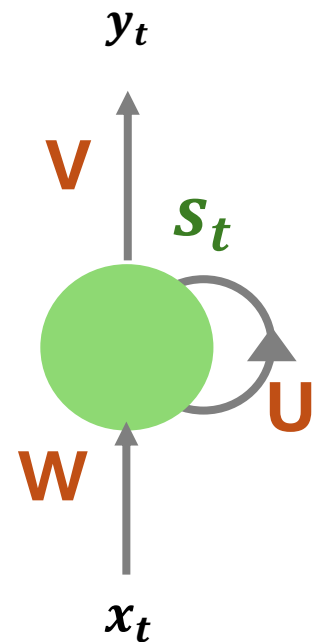
- Word Embedding
 - Word2Vec (CBOW)
 - Evaluation
 - ELMo
- Seq2Seq – Machine Translation
- Attention
- Transformer
 - Architecture
 - Inference (translation)
 - Training (translation, teacher-forcing)
 - Architecture walkthrough (AW)
 - Attention step-by-step
 - Summary
- BERT
 - Architecture and pre-training
 - Using BERT for word embeddings
 - Using BERT for other NLP tasks
 - Tokenization
 - Putting BERT into context
- BART - another Pre-trained Language Model Architecture
- GPT ...

Revision - RNN

x_t : input at time t
 s_t : state at time t
 y_t : output, if any, at time t
 U, V, W : weight matrices

$$s_t = g(Wx_t + Us_{t-1})$$
$$y_t = g(Vs_t)$$

**biases left out to simplify*



RNNs good for input in the form of a **sequence**.

- RNN can process input of any length with fixed model size

Variants: Long Short-Term Memory (LSTM), 2-layer RNN,
Multi-layer RNN, bidirectional RNN,

Applications:

- Named Entity Recognition (NER) (Many-to-Many)
- Sentiment Analysis (Many-to-One)
- Language Model (One-to-Many)
- Machine Translation (Many-to-Many)

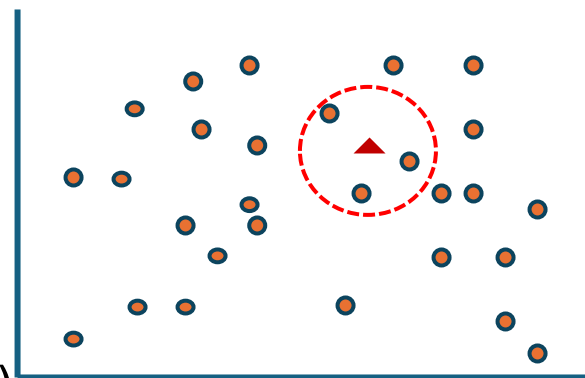
Revision – Word Representation

Language Applications require good word representation which should be able to capture meaning of words.

- Meaning of some Chinese words can be deduced through their representations, e.g.,
日,月,明,上,下,一,二,三,尖,...海,湖,溪,河...木,林,树,森...金,铜,铁,钱...

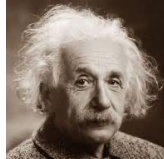
Vector Space Models (VSM)

- SMART library, Taobao recommender
-- mapping books/products to a vector
- Similar books/products are close in vector space
- Answer to query (represented by vector▲) are books/products close to query



A **word embedding** is a learned representation in the form of a **real-valued vector** that encodes the “**meaning**” of the word (i.e., **similarity, difference, semantic relationship**) such that the words that are closer in the vector space are similar in meaning.

Revision – Word Vectors (Embedding)

								...
	King	Queen	Prince	Gong Li	Einstein	dress	shampoo	
Royalty	0.99	0.99	0.98	0.02	0.04	0.5	0.5	
Gender	0.99	- 1.0	0.98	-0.9	0.97	-0.7	-0.1	
Wealth	0.9	0.87	0.5	0.6	0.2	0.6	0.35	
Age	0.76	0.78	0.38	0.59	- 1	0.3	0.45	...
Nationality	0.8	0.8	0.8	0.4	0.7	-0.5	-0.2	
Hair	0.3	0.6	0.45	0.85	0.65	0.1	0.98	
	...	...	...	...	...	...	...	

- **Word vector “embedding”** is a point in a multi-dimensional space, where each dimension represents a particular **feature** (aspect or characteristic) of the word
- **Learning word meaning** is to map words to points (vectors).

*How to construct the word vectors (features and values)
for all words?*

- **DL learns (end-to-end) both features and corresponding values** in the multi-dimensional space.

Word Embedding: Word2Vec by Continuous Bag of Words (CBOW) Model

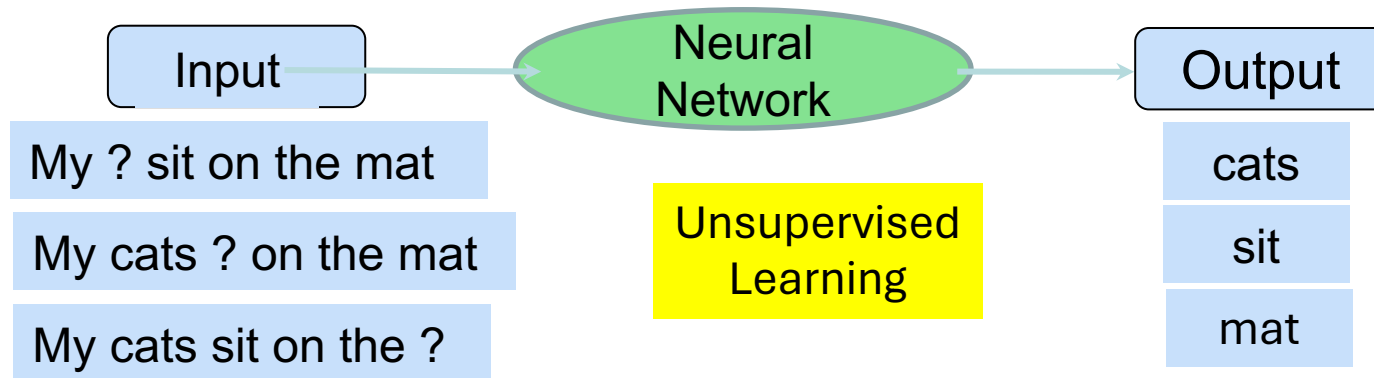
Distributional hypothesis: you know a word by the company it keeps

Given context (or surrounding words), **predict the middle word**.

Example: Guess ? from {"My", "cats", ?, "on", "the", "mat"}

Possible words ("sit", "sat", "sleep", "jump", "crawl", "pee", "walked"...)

Definitely not ("iron", "whale", "north", "happy", ...)



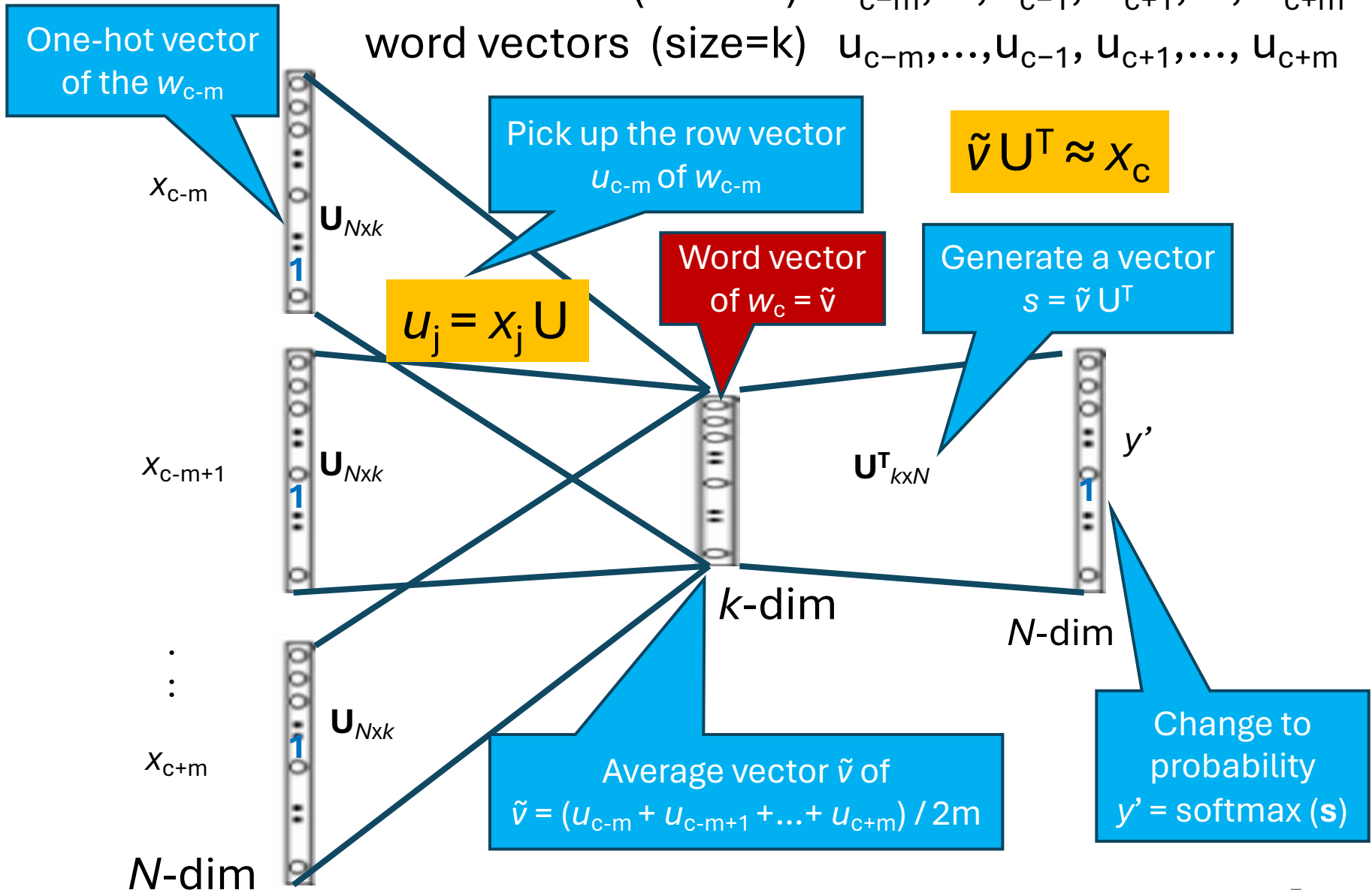
What are the training data?

All documents

Train DL network with many documents and output the center word within each window, e.g., "cats", "sit", "mat"

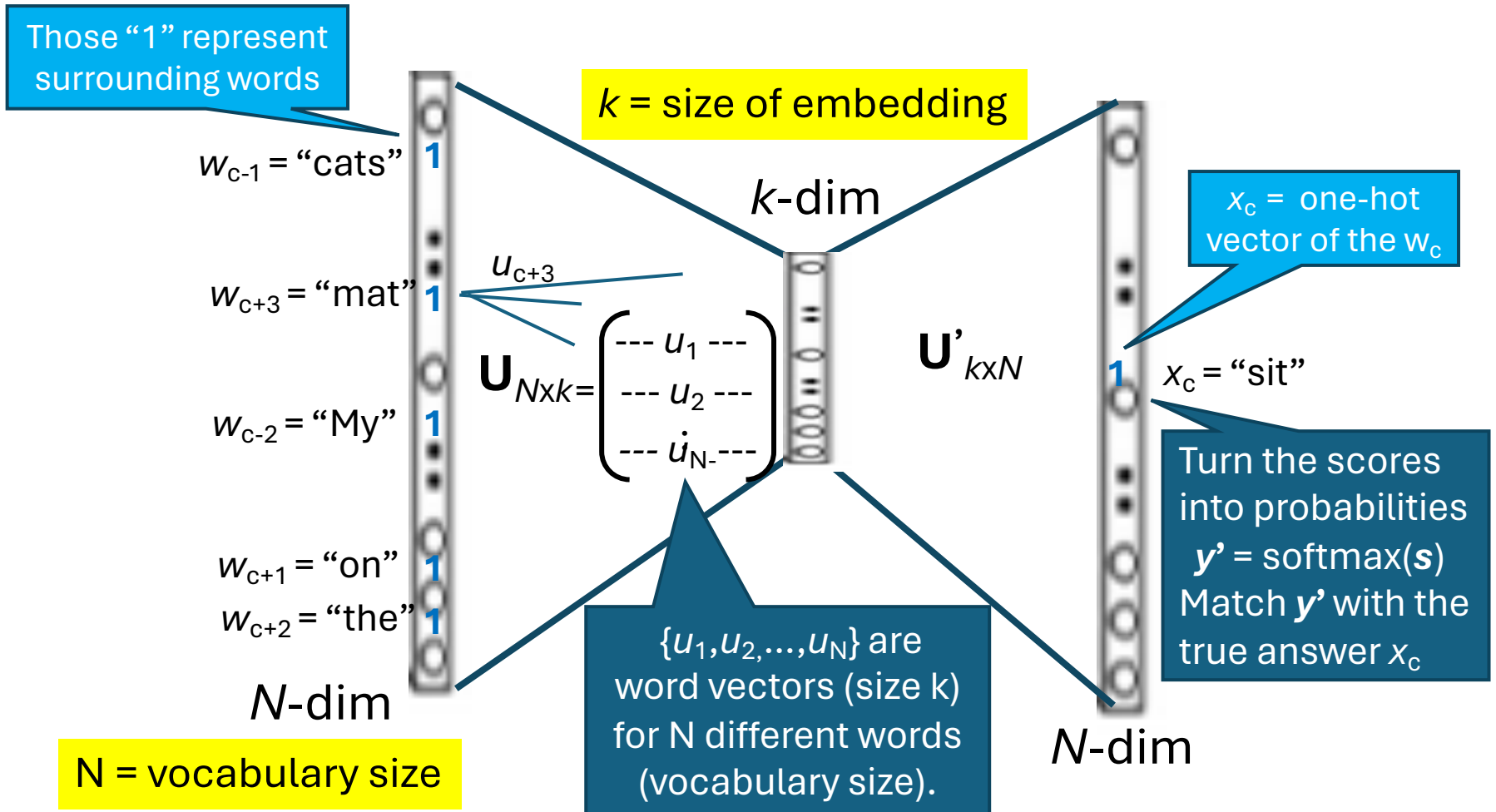
Observations: Similar words, "smart" and "intelligent" likely to have similar context, and similar word vectors.

Window size = m , Predict w_c based on $w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m}$
 one hot-vectors (size= N) $x_{c-m}, \dots, x_{c-1}, x_{c+1}, \dots, x_{c+m}$
 word vectors (size= k) $u_{c-m}, \dots, u_{c-1}, u_{c+1}, \dots, u_{c+m}$



Word Embedding: Word2Vec by CBOW

- Predict center word “sit” from surrounding context {“My”, “cats”, “??”, “on”, “the”, “mat”}



An Example

Ref.: Various Sources from the Internet

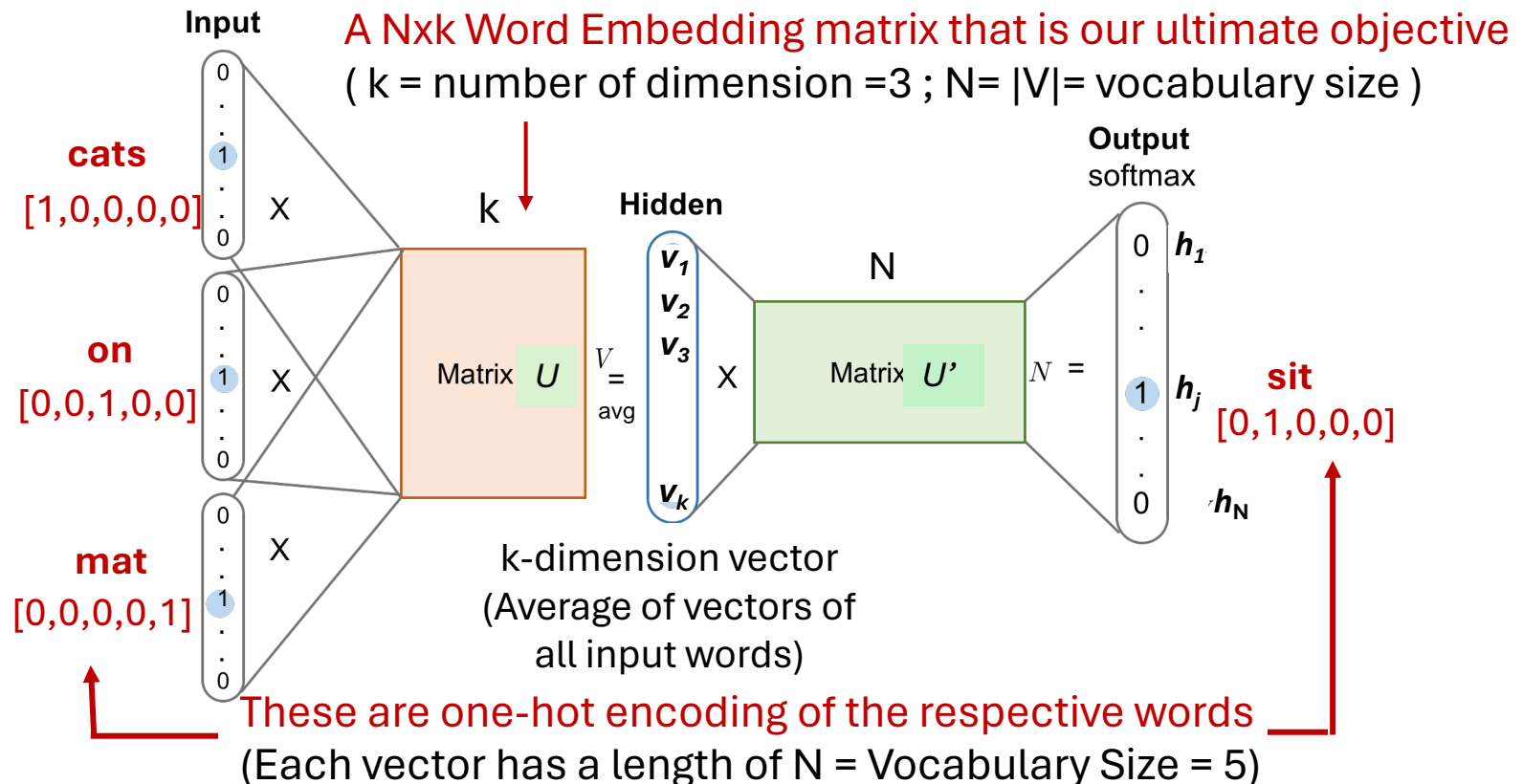
Contributor: Isaac Mak

Example: Guess ? from {“cat”, ?, “on”, “mat”}

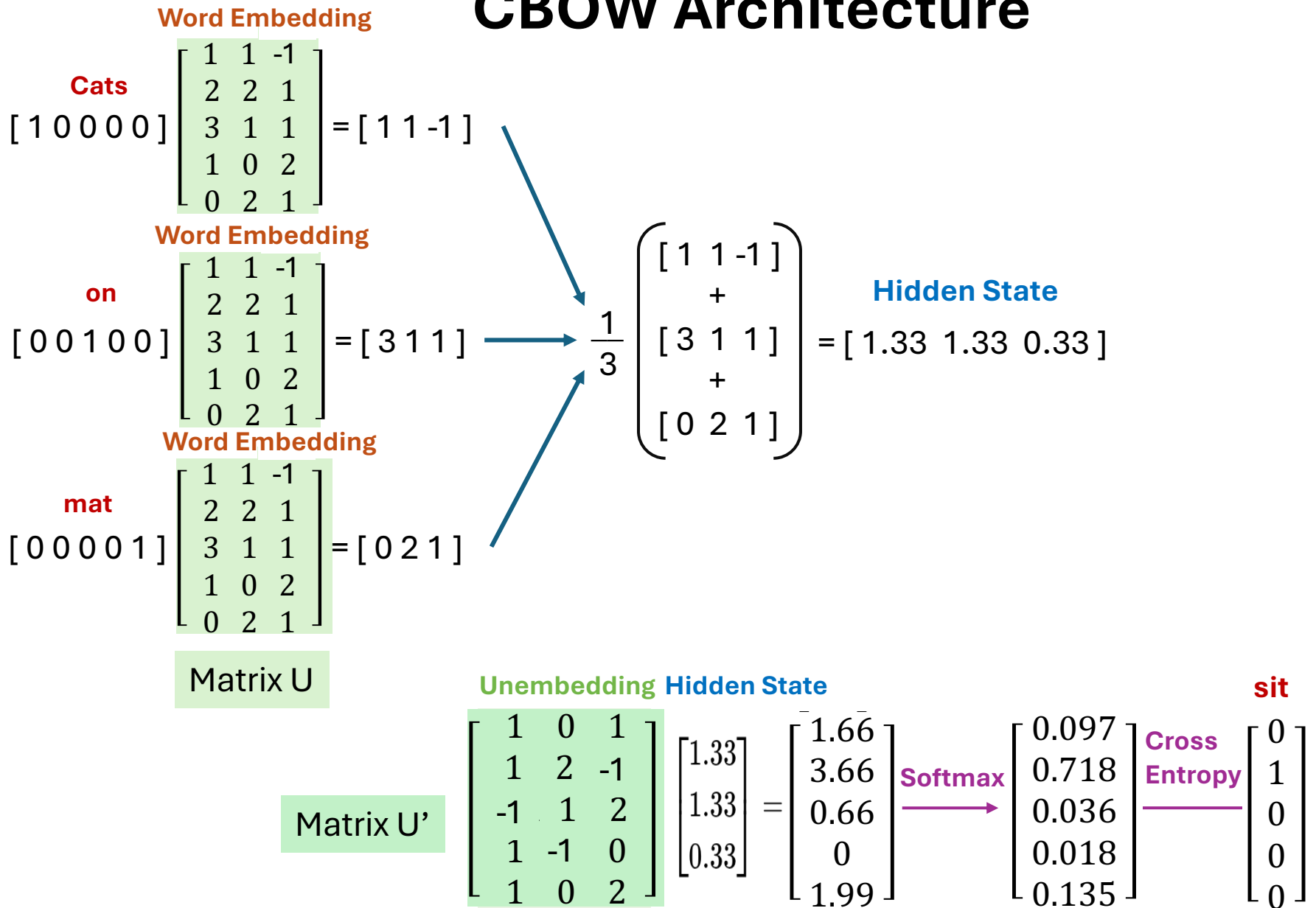
$V = \{ \text{“cats”, “sit”, “on”, “cloud”, “mat”} \}$

1-hot vector (1,0,0,0,0) (0,1,0,0,0) (0,0,1,0,0) (0,0,0,1,0) (0,0,0,0,1)

Vocabulary size $|V| = 5$; $k = \text{size of the word vector} = 3$



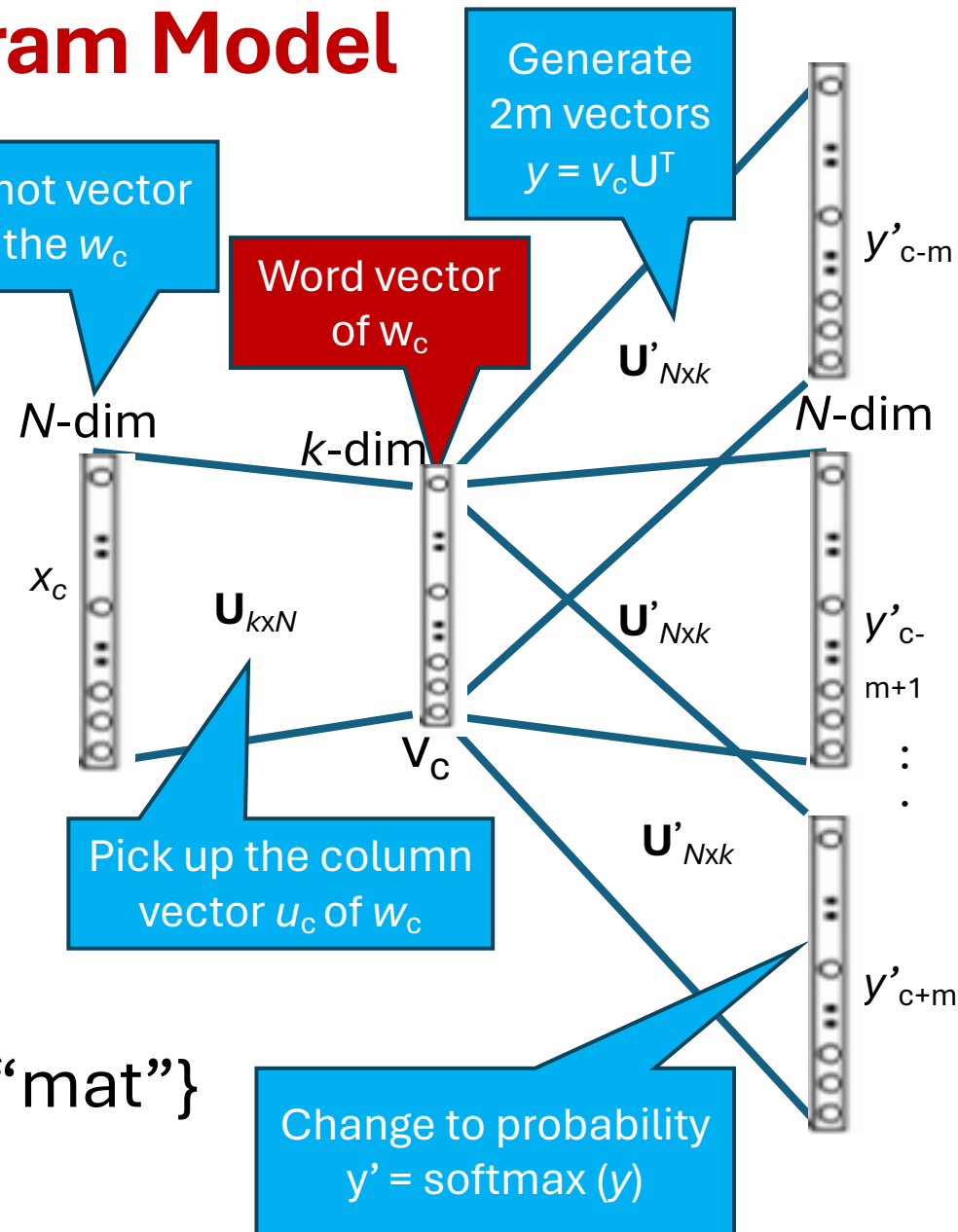
CBOW Architecture



Word2Vec by Skip-Gram Model

In the **Skip-gram** model, the input word is used to **predict the context**.

Given center word “sit”,
predict the surrounding
words
{“My”, “cats”, “on”, “the”, “mat”}



Word Embeddings: Evaluation

Quality of word vectors depends on:

- Depends on model (co-occurrence count, CBOW, ...)
- Size of the embedding vector
performance is lower for extremely low
as well as for extremely high dimensional word vectors
- Window size
- Training data, corpus (size and domain)
performance increases with larger corpus sizes

Problems

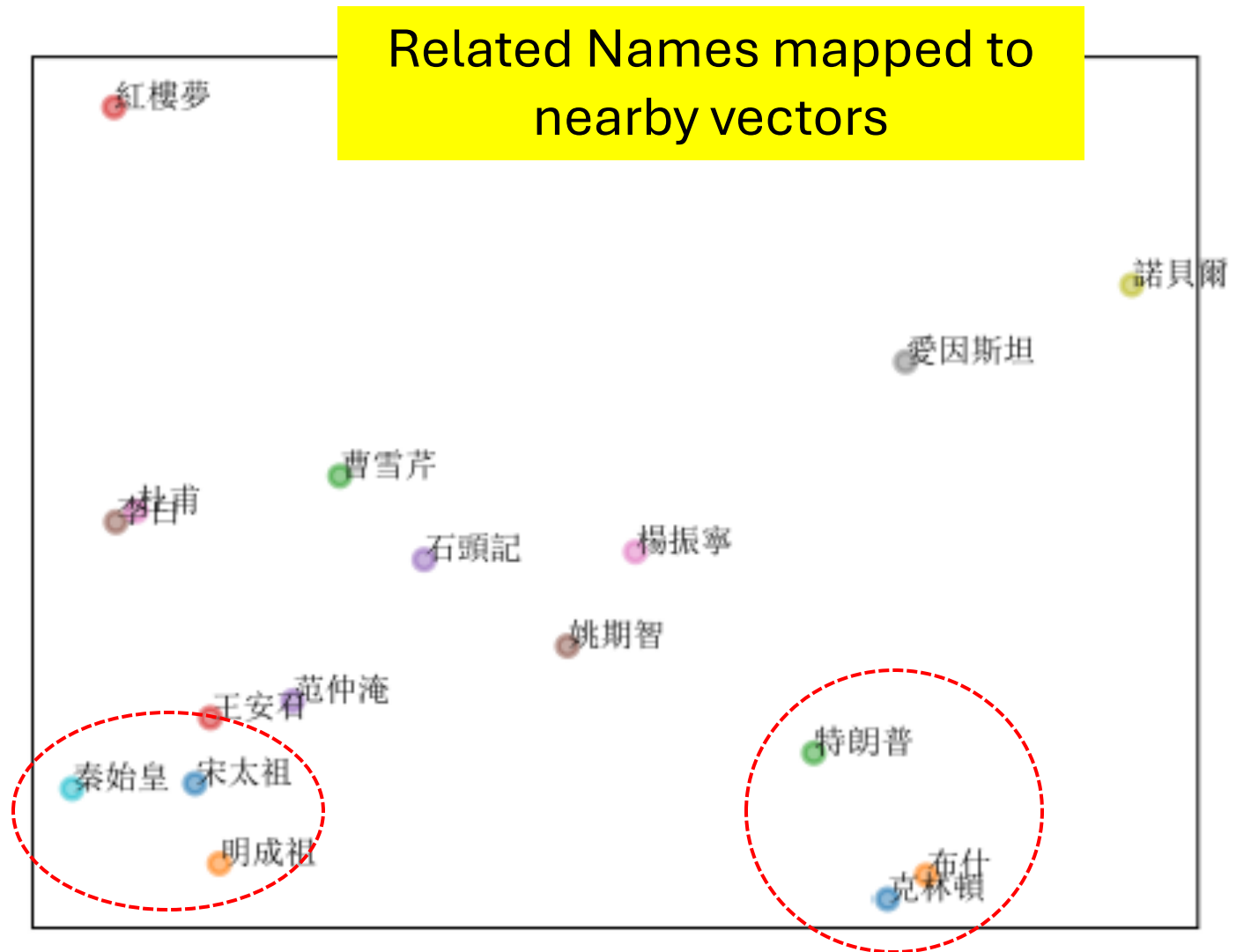
- Ambiguous words: e.g., share, run ... (verb and noun)

Evaluation

- Intrinsic: word similarity, word analogy
- Extrinsic

Word Embeddings: Similarity

Training with over 4 million documents.

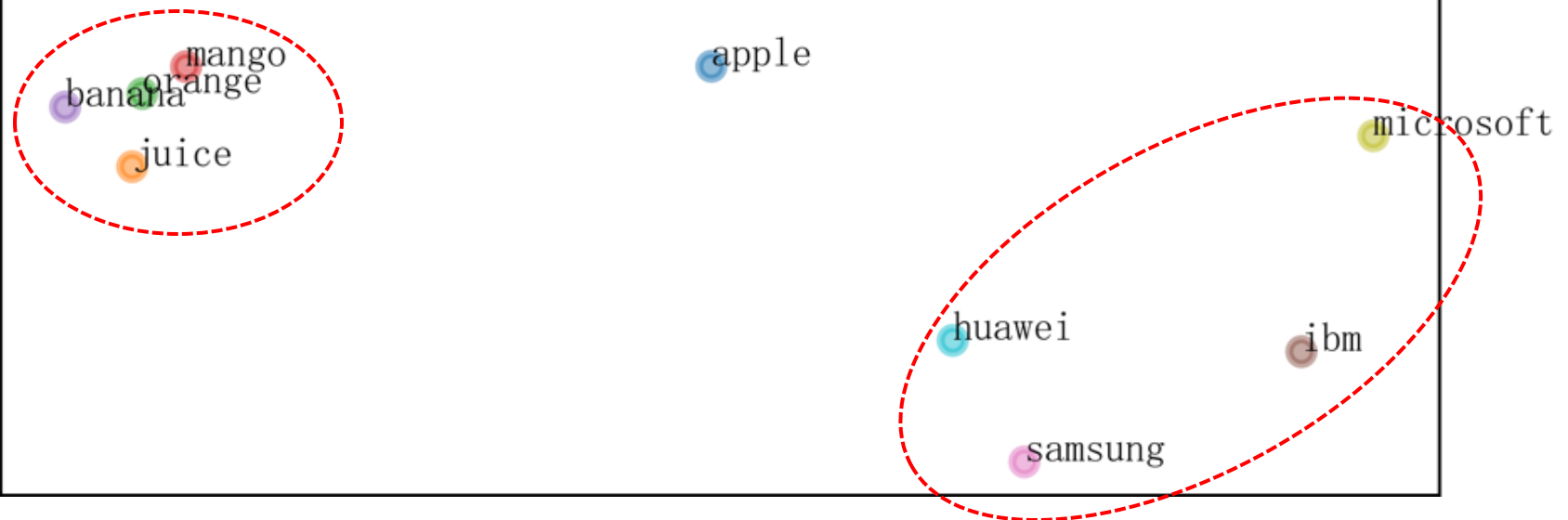


Word Embeddings: Word Similarity

```
1 model = gensim.models.Word2Vec.load(os.path.join(MODEL_PATH, 'word2vec.model'), mmap='r')
2 print(model.similarity('蘇軾', '東坡'))
3 0.612037855344
4
5 pprint(model.most_similar('奧斯卡'))
6 [( '奧斯卡金像獎', 0.6850196123123169),
7  ( '奧斯卡獎', 0.6607037782669067),
8  ( '金球獎', 0.6433079242706299),
9  ( '東尼獎', 0.5794373750686646),
10 ( '金棕櫚獎', 0.5717018842697144),
11 ( '艾美獎', 0.5554369688034058),
12 ( '斷背山', 0.5518748760223389),
13 ( '泰坦尼克號', 0.5482996702194214),
14 ( '布克獎', 0.546217679977417),
15 ( '學院獎', 0.5445188879966736)]
```

Words with similar meaning
mapped to nearby vectors

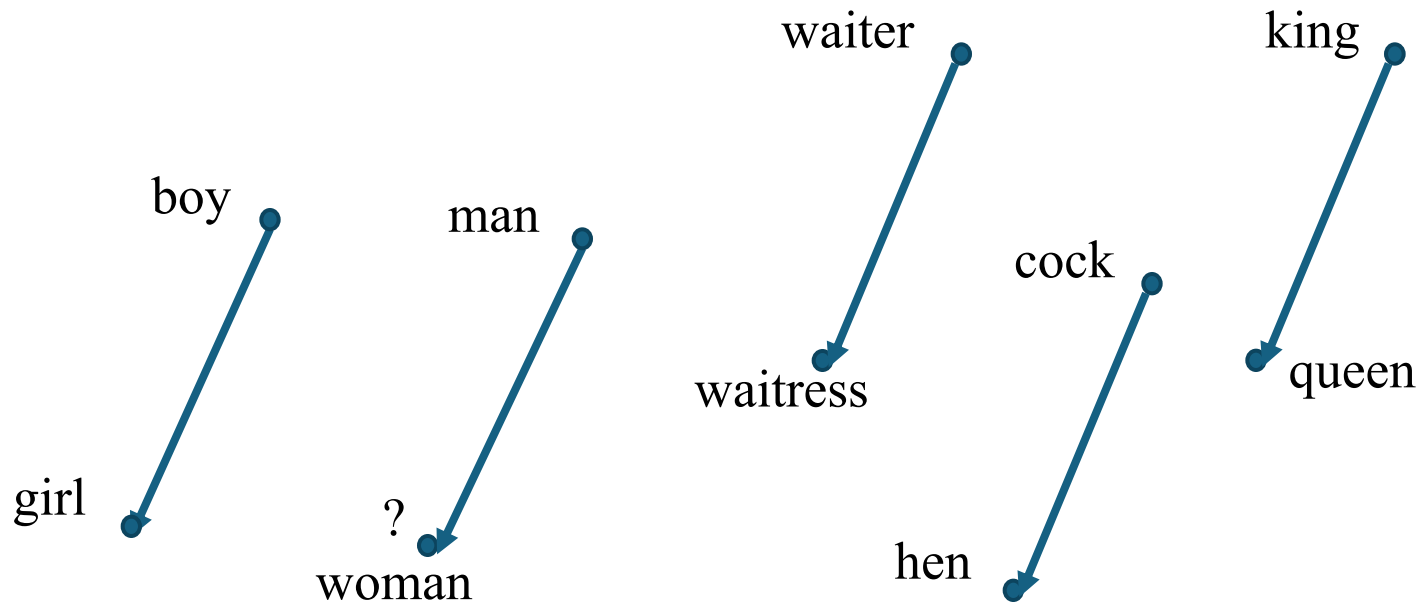
Word vector depends on
context, generated at run time
(e.g., ELMo, Transformer)



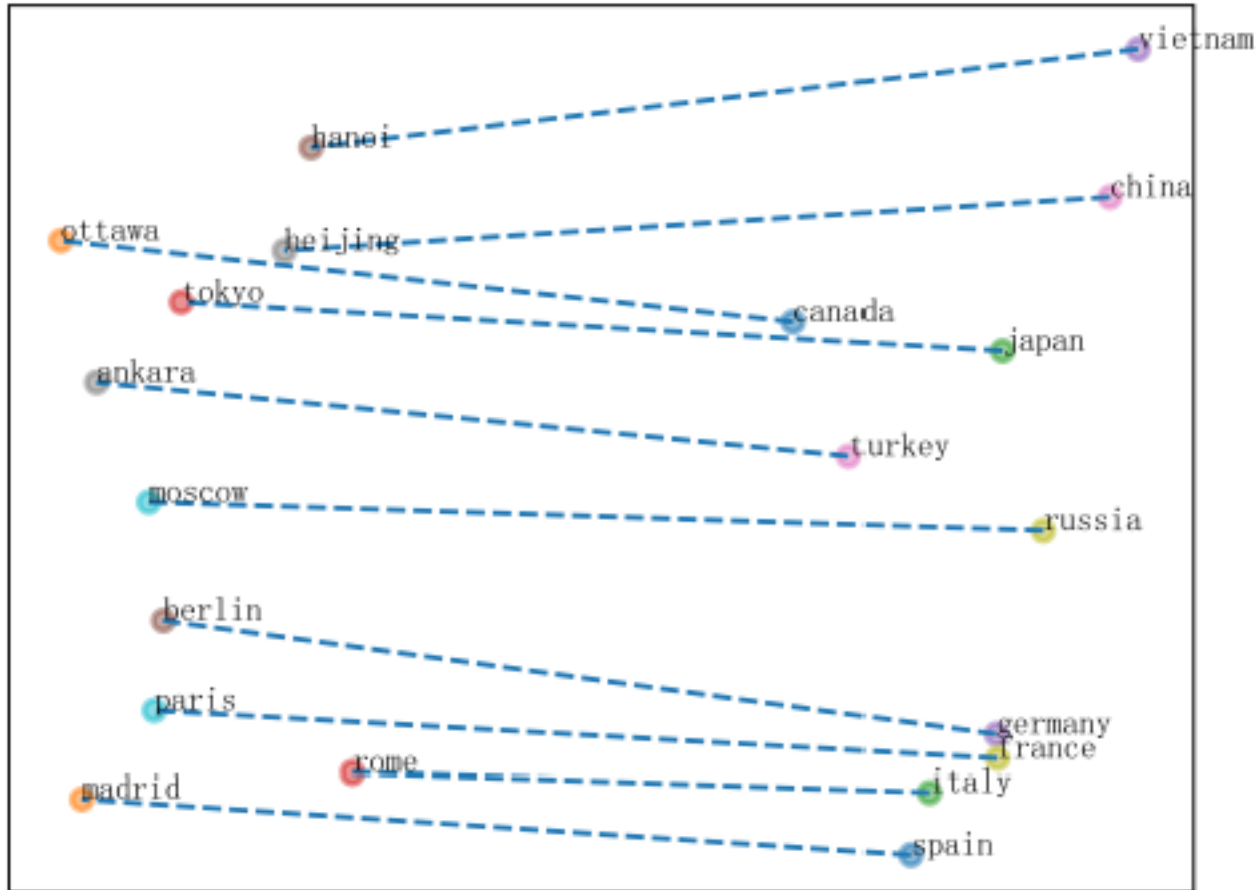
Word Analogies: Gender

boy : girl
= man : woman
= king : queen
= cock : hen
= waiter : waitress

Pairs of words with the same relation are connected by the similar vector
These vectors are similar and represent the gender



Word Analogy: Country-Capital Pairs



Word Analogy: superlatives & tenses

Superlative adjectives

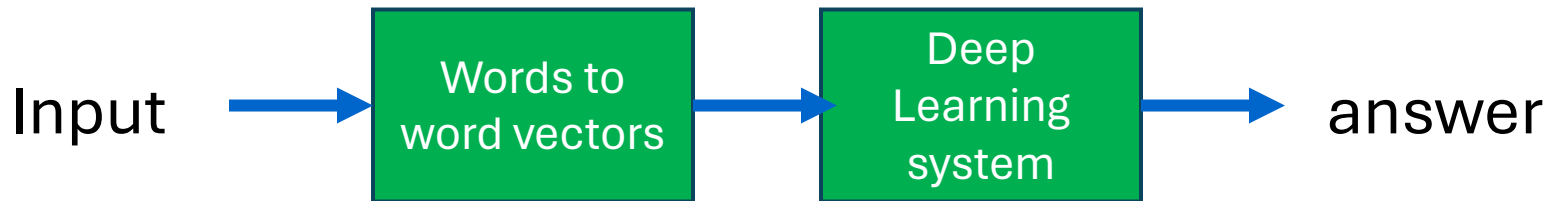
- good : best = big : biggest
- good : best = bad : worst
- good : best = cold : coldest
- good : best = easy : easiest

Present and past tense

- walking : walked = increasing : increased
- walking : walked = falling : fell
- walking : walked = going : went
- walking : walked = hitting : hit
- walking : walked = hiding : hid

Word Embeddings: Extrinsic Evaluation

Extrinsic evaluation: To evaluate the performance of word representations in downstream tasks



- Examples of tasks:
 - Name entity recognition (date, currency, address, names, organization)
 - Sentiment classification
 - Part of speech
 - Questions and answers system
- Shortcomings:
 - Is the evaluation on a real task?
 - Slow to compute performance
 - Unclear if subsystem is the problem, other subsystems, or internal interactions?

Embeddings from Language Models (ELMo)

<https://allenai.org/allennlp/software/elmo> ; <https://www.youtube.com/watch?v=csAlW9HmwAQ>



What and Why ELMo?

Unlike other word embeddings, ELMo is a **computational model** that generates word vectors on run time.

- **Context-sensitive:** Embedding depends on word's context

I can't **trust** you.

He has no **trust** for his friends.

He has a **trust** fund.

I let the kids outside to **play**.

It wasn't a **play** the coach would approve of.

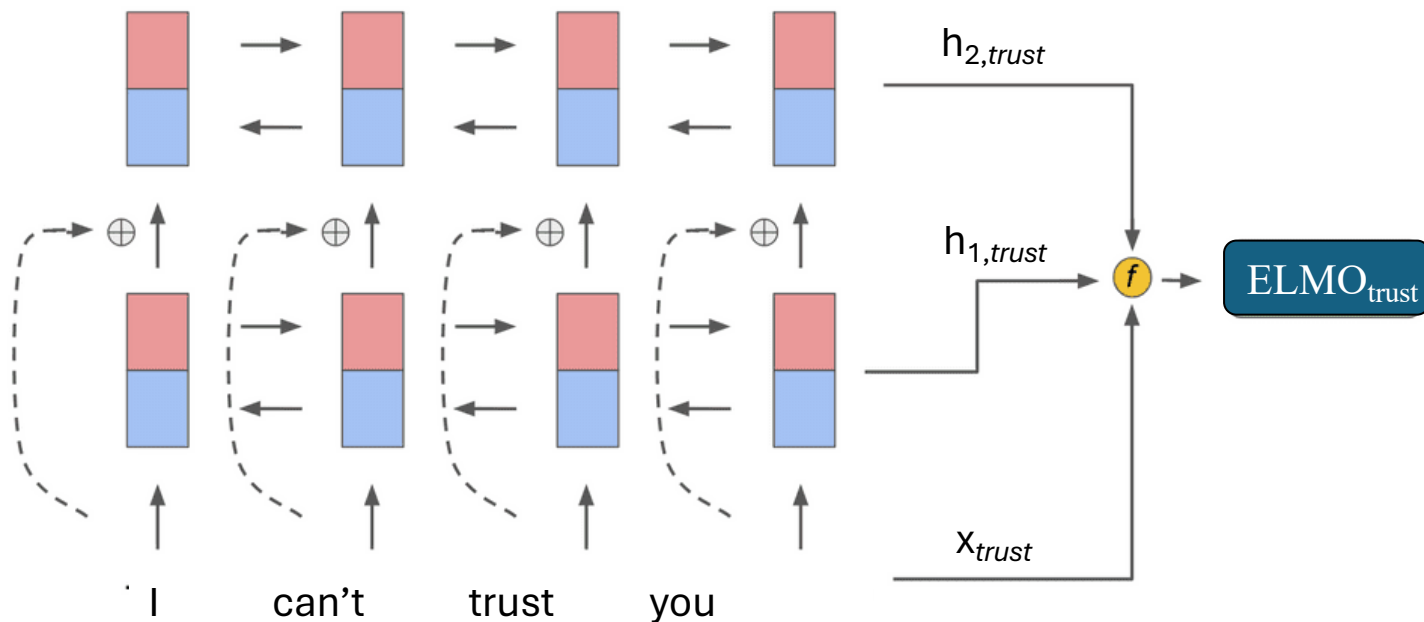
He acted in a famous **play**.

Different word vectors for “trust” and “play” in these examples

- **Deep:** representations combine all layers of the network
- **Trained on lots of data:** 5 billion words (unannotated)
- **Open-source**

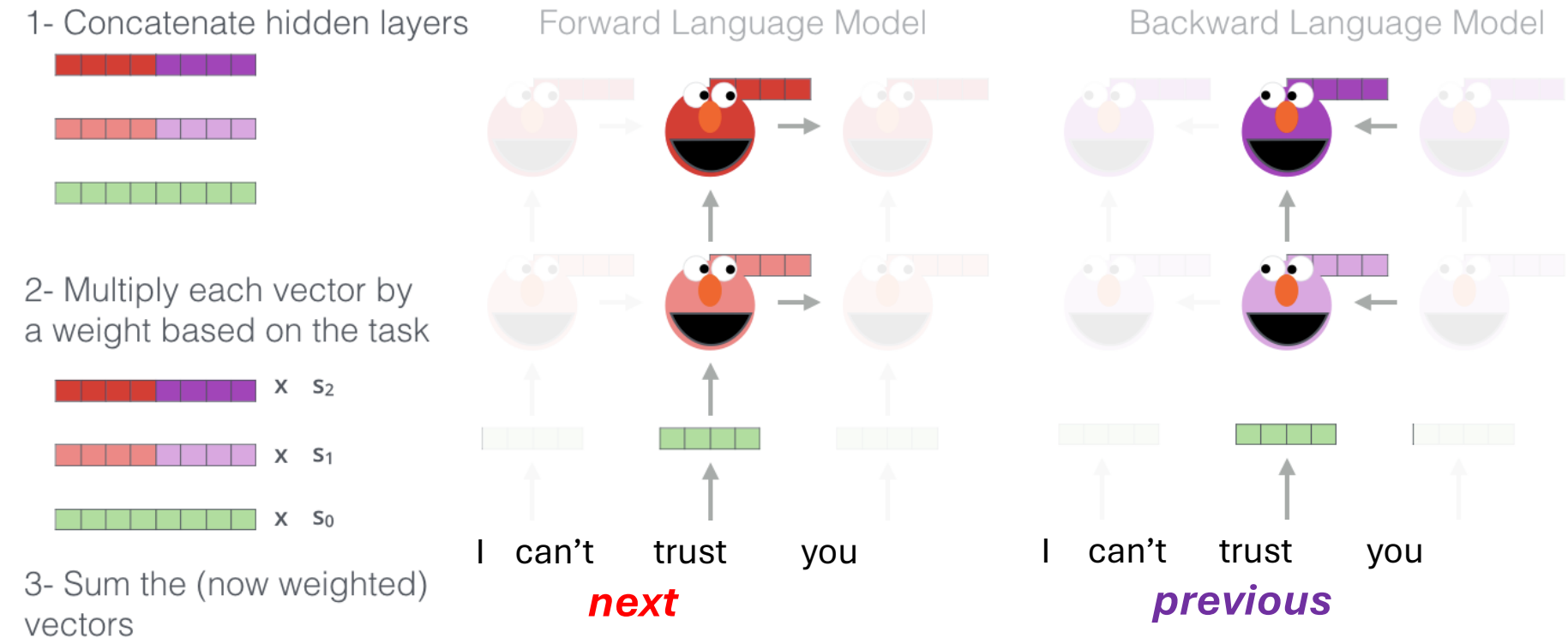
ELMO Model

- ELMO uses a 2-layer bidirectional LSTM backbone
- Residual connection between the first and second layers.
- ELMO combines (1) word representation x_n ,
(2) bidirectional hidden layer representations $h_{1,n}$ and $h_{2,n}$
into a new weighted word representation for the n^{th} word.



Embedding of “trust” in “I can’t trust you”

- ELMo was trained to predict the next and previous words in a sequence of words, e.g., “I can’t trust you”.



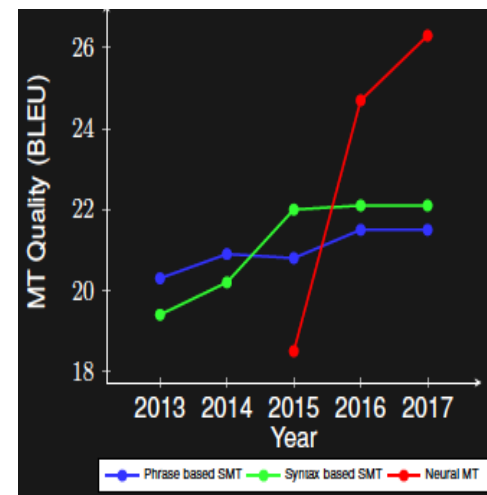
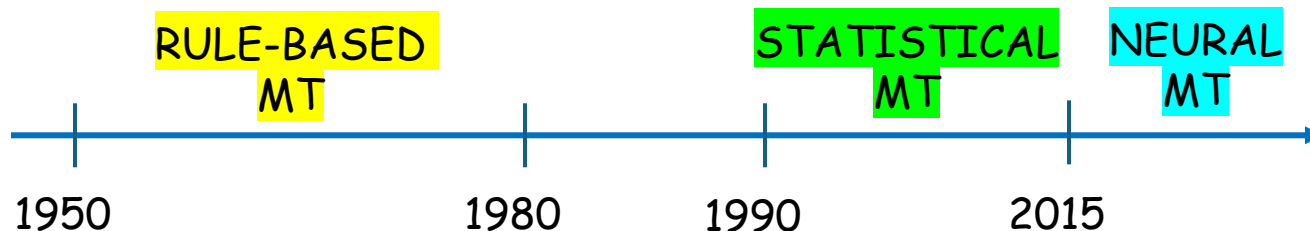
$$\text{ELMo}_k = s_0 x_k + s_1 h_{1,k} + s_2 h_{2,k}$$

ELMo learns a separate representation for each task, e.g., Question answering, sentiment analysis, etc.

NLP (1) – Outline

- Word Embedding
 - Word2Vec (CBOW)
 - Evaluation
 - ELMo
- Seq2Seq – Machine Translation
- Attention
- Transformer
 - Architecture
 - Inference (translation)
 - Training (translation, teacher-forcing)
 - Architecture walkthrough (AW)
 - Attention step-by-step
 - Summary
- BERT
 - Architecture and pre-training
 - Using BERT for word embeddings
 - Using BERT for other NLP tasks
 - Tokenization
 - Putting BERT into context
- BART - another Pre-trained Language Model Architecture
- GPT ...

MACHINE TRANSLATION



RULE-BASED MT

Translate :

- I want to go to the market today.

Break sentence into chunks

- I | want | to go | to the market | today.

Translate model

- 我 | 要 | 去 | 市場 | 今天

Language model

Rule based : adding rule to reorder words

- 我今天要去市場 or 今天我要去市場

STATISTICAL MT

Generate all possible sentences

我 要求 走 交易 今天
我 會 消失 市場 現在
消失 我 會 今天 市場

...

Find most common similar one from statistics or language model

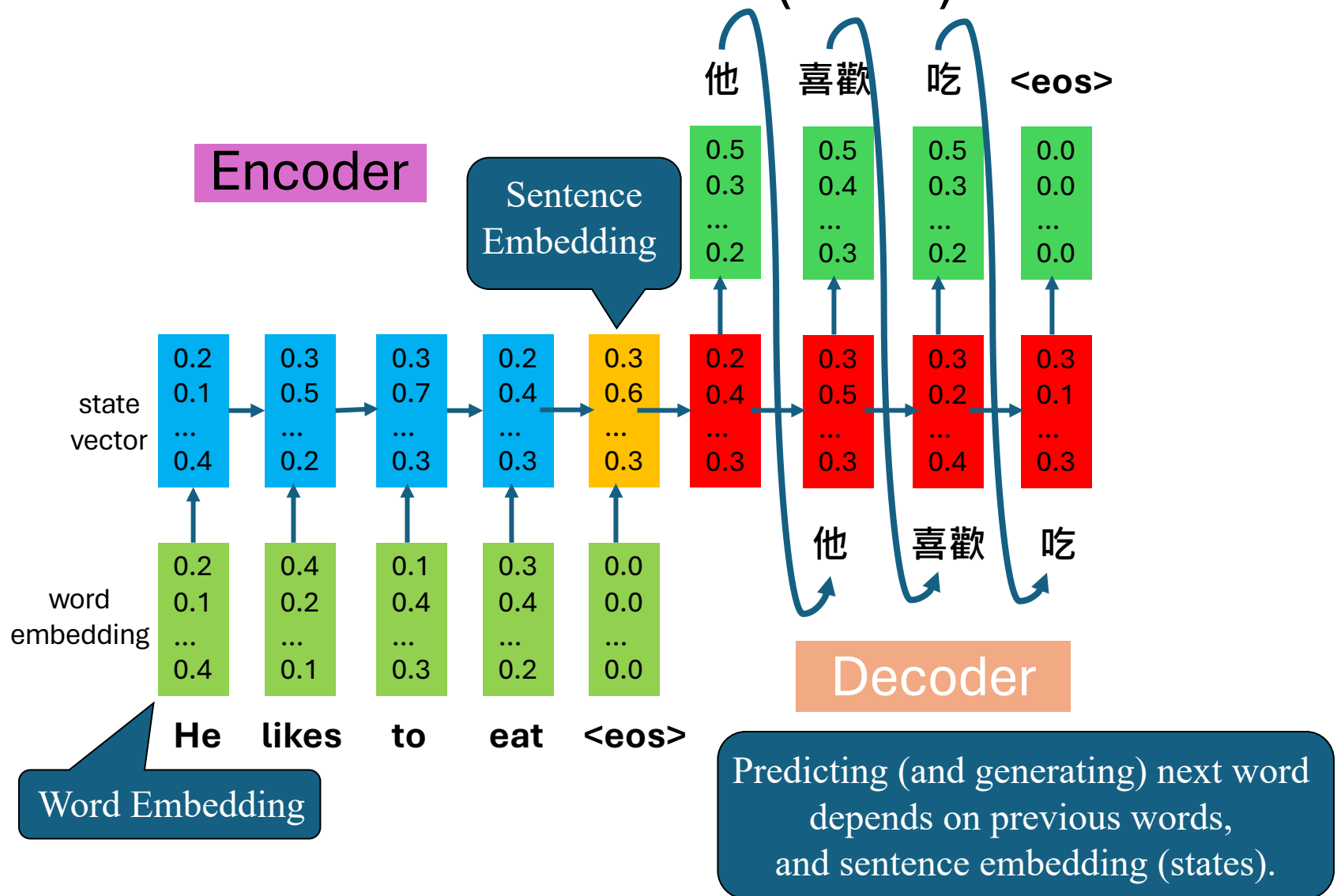
今天 我 要 去 市場
我 今天 需要 去 市場

Neural MT

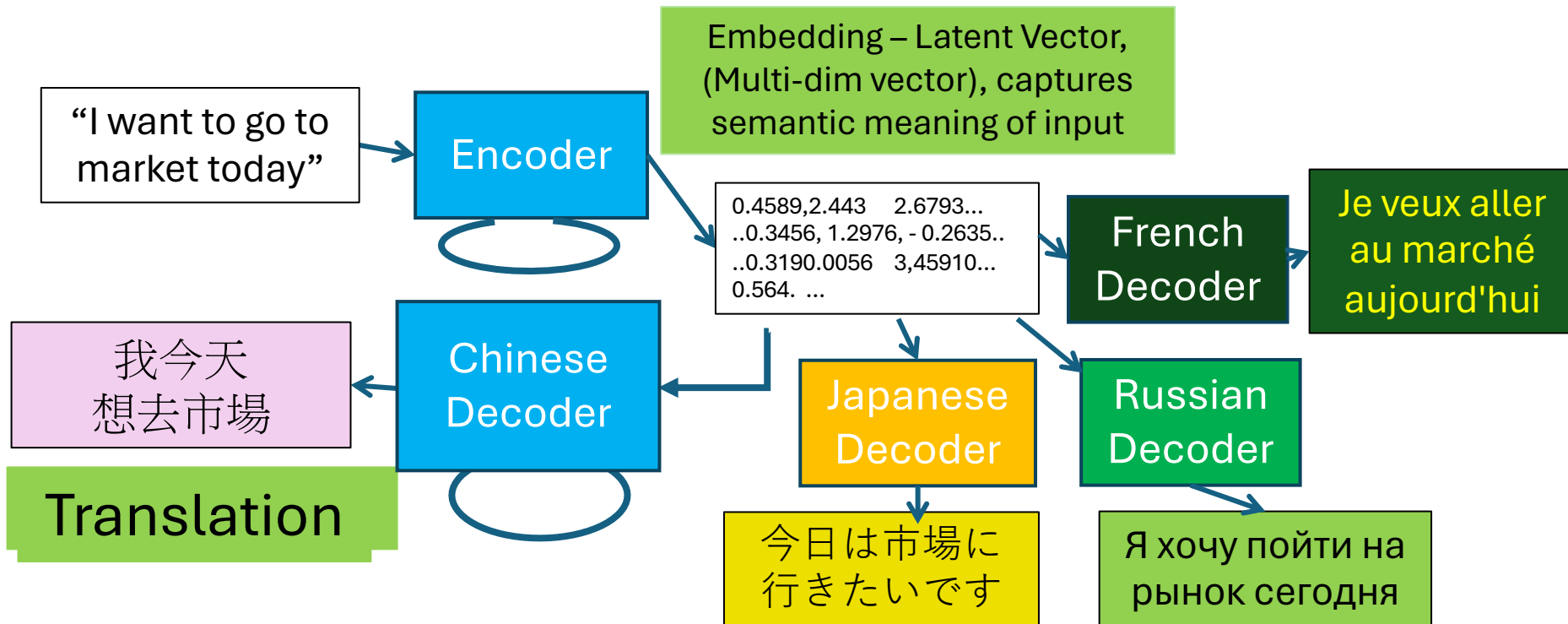
Sequence to Sequence Learning with Neural Networks (Sutskever, Vinyals, Le, 2014)

Sequence to Sequence Translation

Encoder-Decoder Model (2014)



Machine Translation

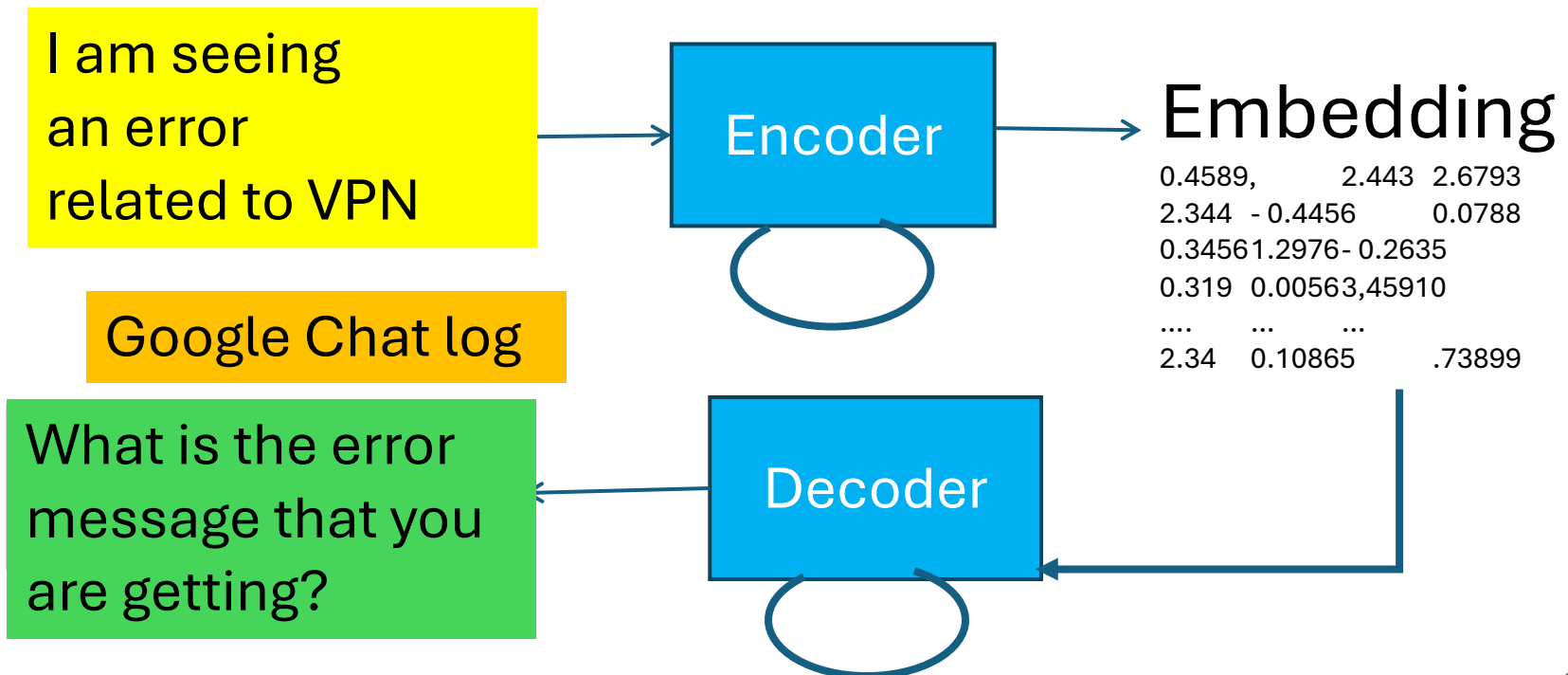


Trained with paired sentences

Automatic Answering System

AI Chatbot (same as ChatGPT)

Google Employees and Tech Support Team:



Picture Caption

Input: picture

Output : image caption



Vision Encoder

Embedding

0.4589, 2.443 2.6793
2.344 - 0.4456 0.0788
0.3456 1.2976 - 0.2635
0.319 0.00563, 45910
....
2.34 0.10865 .73899

A white cat
and a red
yarn ball

Text decoder

Attention

“Forget problems” in decoding long sentences

- BiLSTM – bidirectional LSTM
- Encoder-Decoder
- Attention

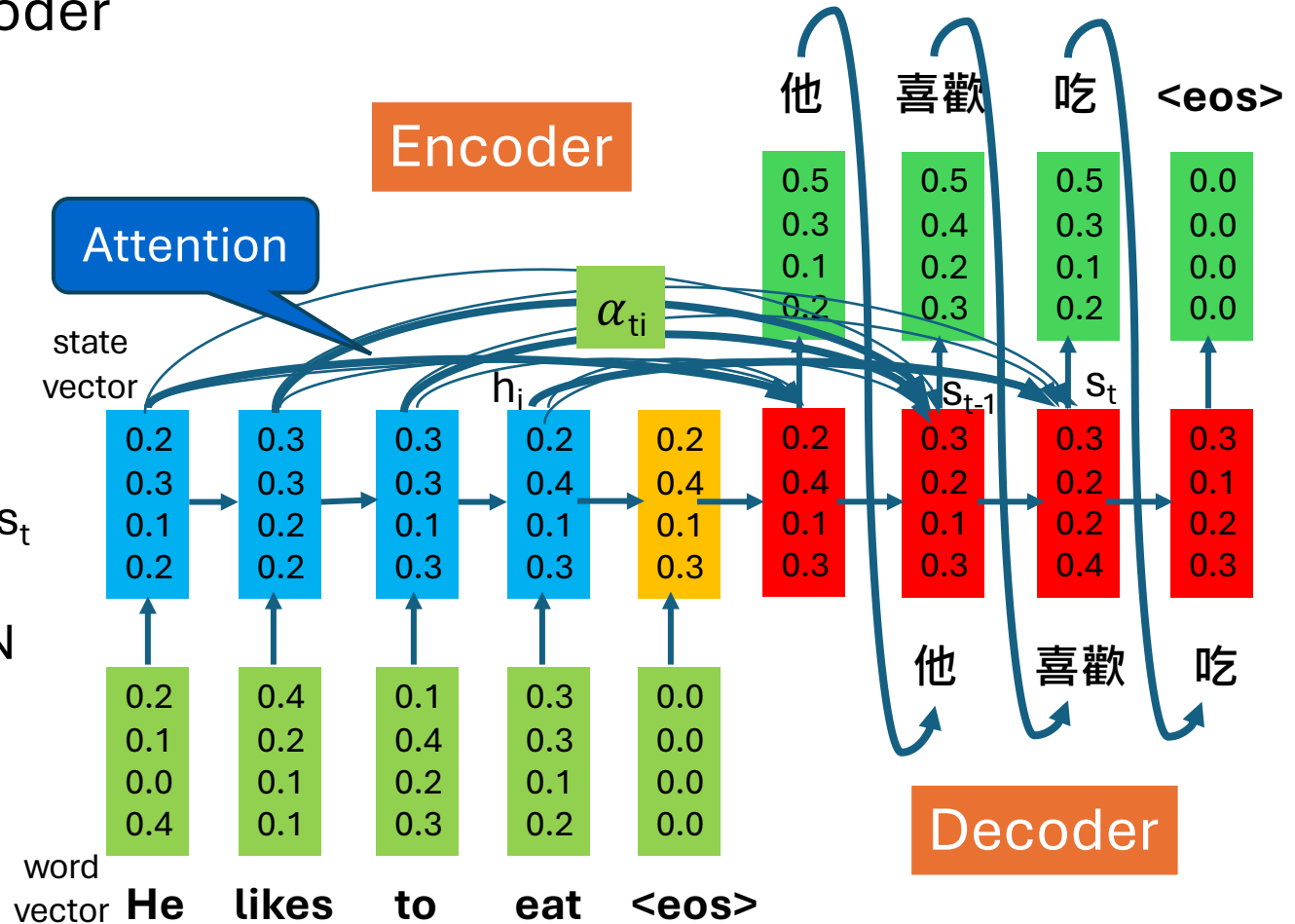
h_i = encoded hidden state

s_t = decoder state

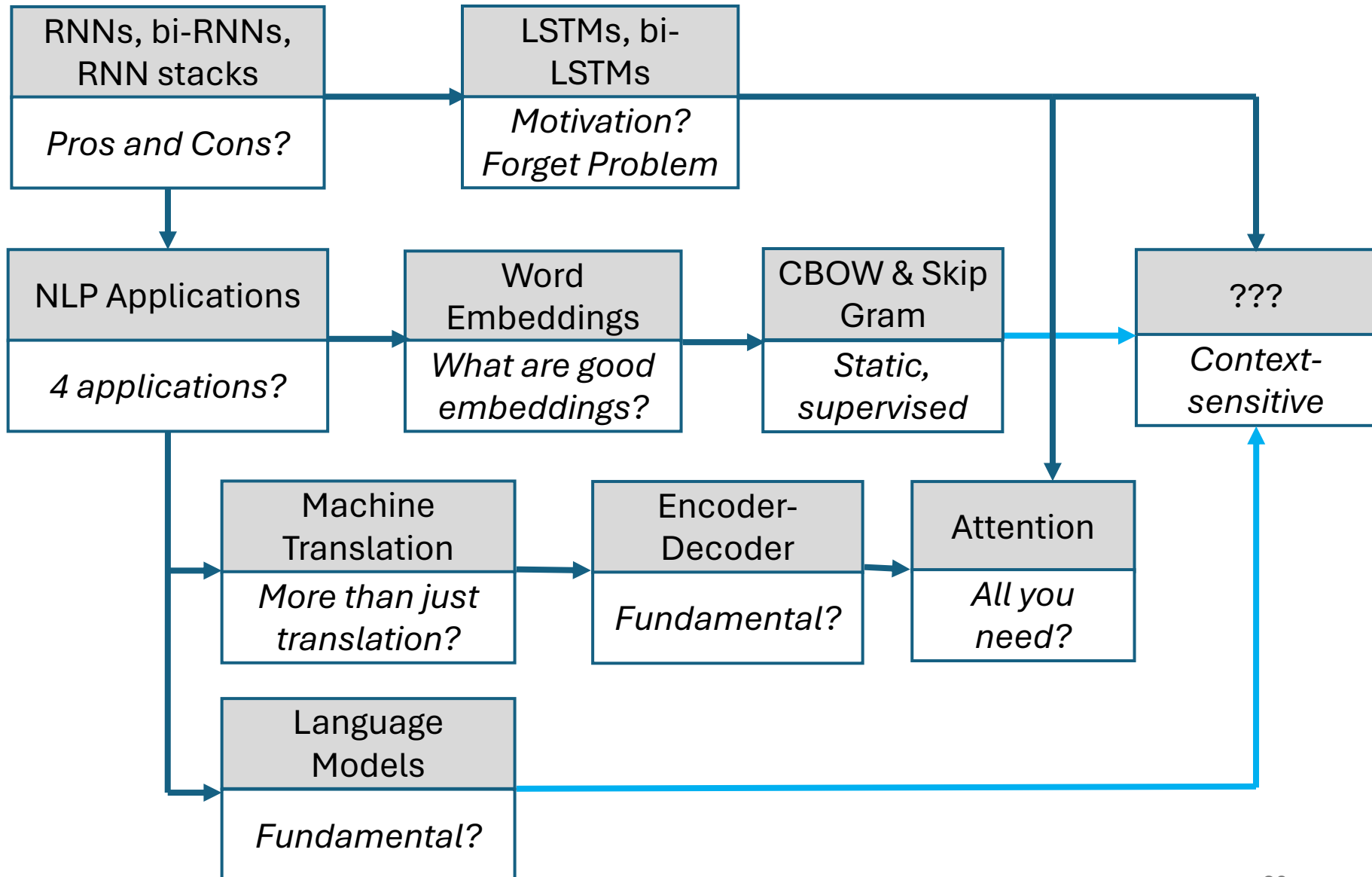
e_{ti} = how well input elements align with s_t
 $= a(s_{t-1}, h_i)$, where
 $a(.)$ = function by NN

Attention weight α_{ti}
 $= \text{softmax}(e_{ti})$

Context vector $\mathbf{c}_t$
 $= \sum_{i=1}^T \alpha_{ti} h_i$



NLP – Mind Map (1)



NLP (2) – Outline

- Word Embedding
 - Word2Vec (CBOW)
 - Evaluation
 - ELMo
- Seq2Seq – Machine Translation
- Attention
- Transformer
 - Architecture
 - Inference (translation)
 - Training (translation, teacher-forcing)
 - Architecture walkthrough (AW)
 - Attention step-by-step
 - Summary
- BERT
 - Architecture and pre-training
 - Using BERT for word embeddings
 - Using BERT for other NLP tasks
 - Tokenization
 - Putting BERT into context
- BART - another Pre-trained Language Model Architecture
- GPT ...

Transformer

A Deep Learning Model for NLP and Computer Vision

Transformer introduced in an influential paper by Google (2017).
Only one author of the transformer paper is still at Google.
Other authors co-founded startups or moved to other companies.

Attention Is All You Need

Adept

Ashish Vaswani*
Google Brain
avaswani@google.com

 character.ai

Noam Shazeer*
Google Brain
noam@google.com

Adept

Niki Parmar*
Google Research
nikip@google.com

 Inceptive

Jakob Uszkoreit*
Google Research
usz@google.com



Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

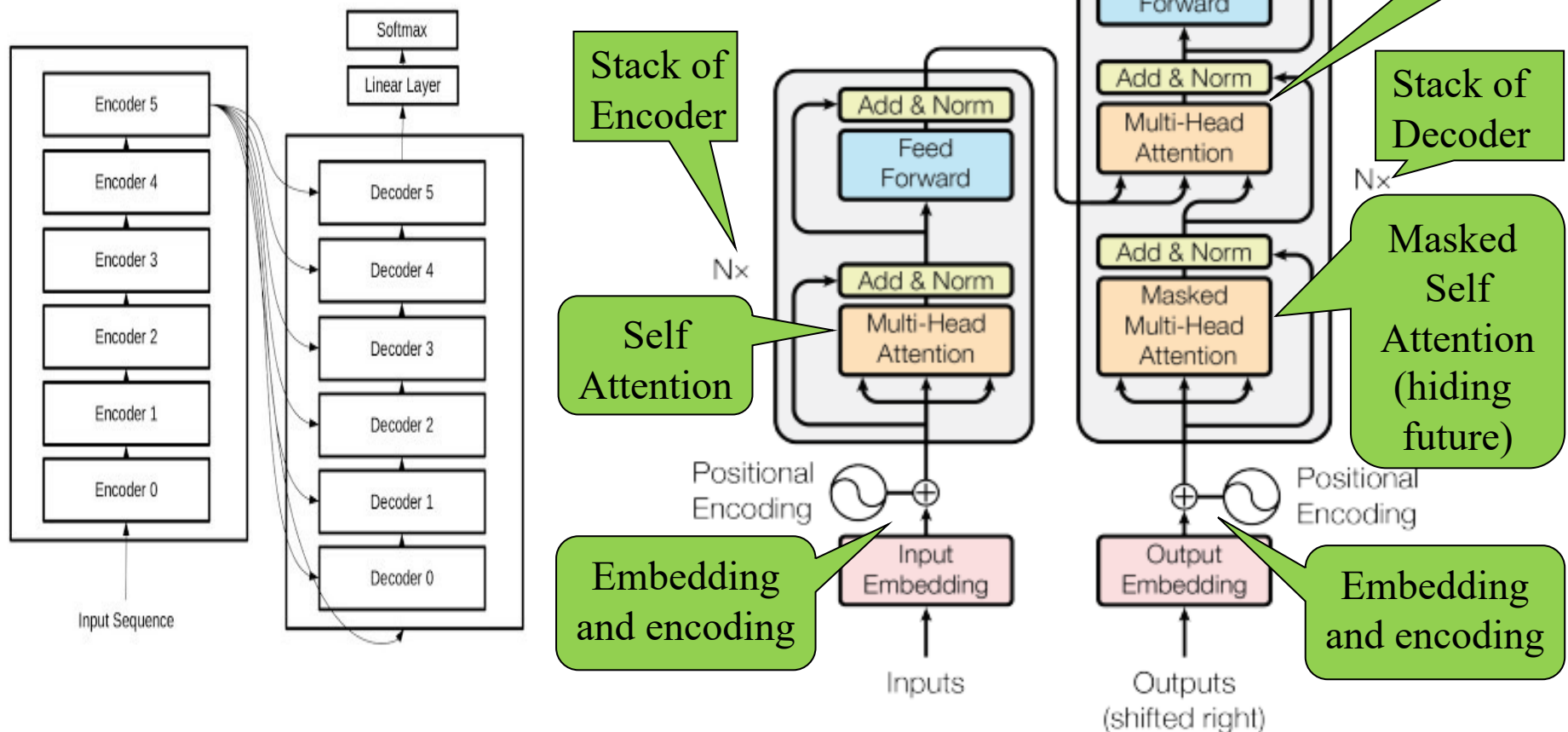


co:here

Illia Polosukhin* †
illia.polosukhin@gmail.com

NEAR
INCORPORATED

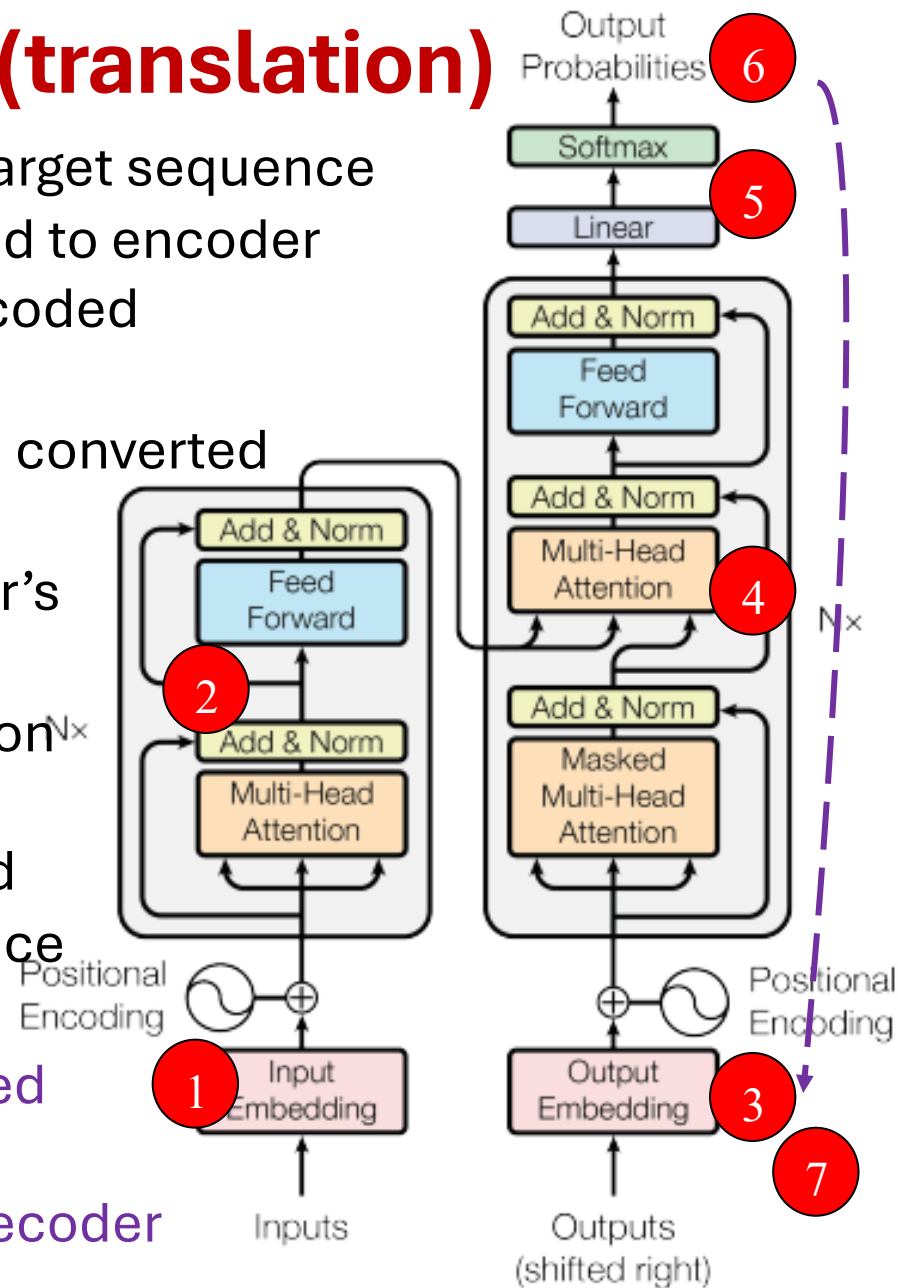
Transformer: Architecture



Transformer: Inference (translation)

Only input sequence is available, no target sequence

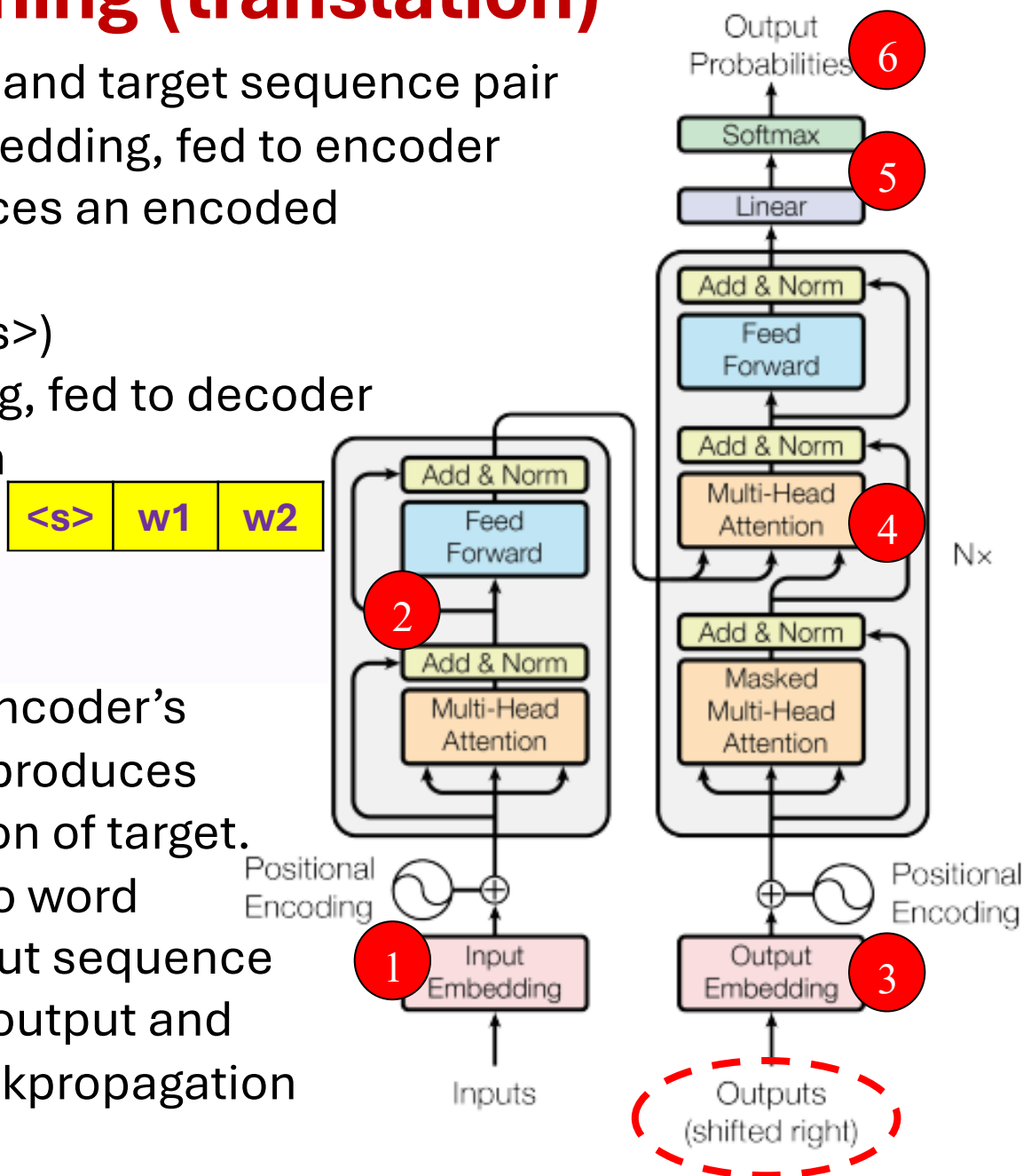
1. Input converted into embedding, fed to encoder
2. Stack of Encoders produces an encoded representation of input
3. 1st position of decoder input is <s>, converted into embedding, fed to decoder
4. Stack of Decoders fed with Encoder's encoded representation of input, produces an encoded representation of target
5. The Output layer converts into word probabilities and an output sequence
6. Take the first word of the output as the predicted word. The word is filled into next position of decoder input
7. Repeat above to fill next position decoder input



Transformer: Training (translation)

Training data: source (input) and target sequence pair

1. Input converted into embedding, fed to encoder
2. Stack of Encoders produces an encoded representation of input
3. Target (prepended with $\langle s \rangle$) converted into embedding, fed to decoder (shifted right one position so that model predicts target word i by seeing only words $1, 2, \dots, i-1$)
4. Stack of Decoders with Encoder's encoded representation produces an encoded representation of target.
5. Output layer converts into word probabilities and an output sequence
6. Loss function compares output and target and is used for Backpropagation



Transformer: Training (Teacher Forcing)

Why can't we train the network as we do the inference?

- That is: Run the Transformer in a loop, take the last word from the output sequence, append it to the Decoder input and feed it to the Decoder for the next iteration (until the end-of-sentence token is predicted).

Reasons:

- training too long
- predict the second word based on a potentially erroneous first predicted word

Act like a Teacher –

- use the correct n^{th} word to predict the $(n+1)^{\text{th}}$ word even though the first word is predicted wrong
- errors don't keep compounding
- greatly speed up training because all the words can be output in parallel without looping

Architecture Walkthrough (AW): Input – Embedding and Positional Encoding

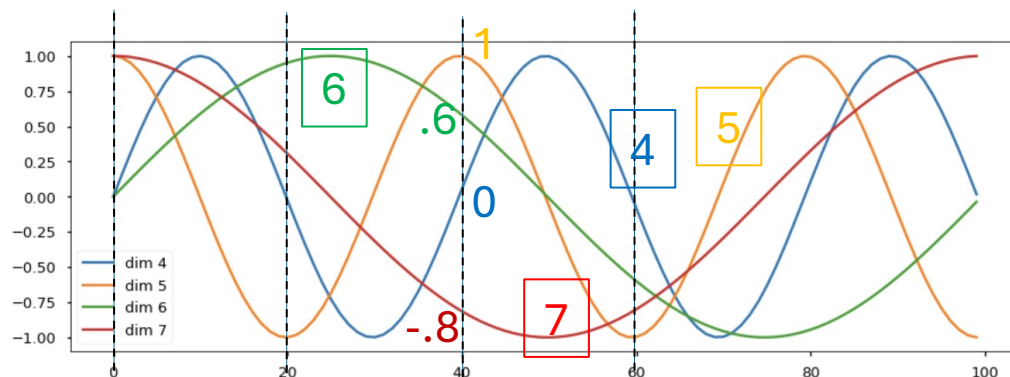
- Word Embedding
encodes word meaning
- Positional Encoding
represents word position (*pos*)
(**different for each position**)

Sinusoidal positional encoding

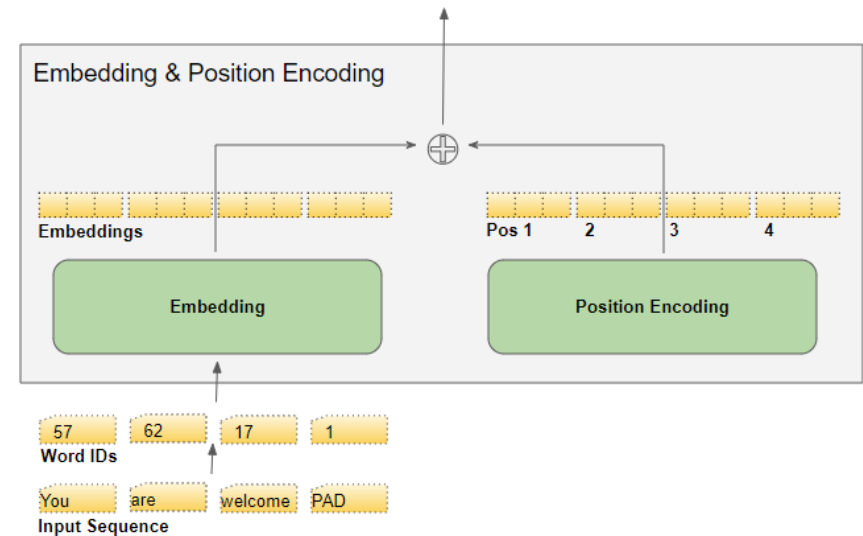
E = word embedding size

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/E})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/E})$$



4 - blue, 5 pink, 6 - green, 7 - red



For example,

E = 512 (implies 512 curves)

i^{th} curve, $0 \leq 2i, 2i+1 < 512$

pos 0 = (... , 0, 1, 0, 1, ...)

pos 20 = (... , 0, -1, .9, .3, ...)

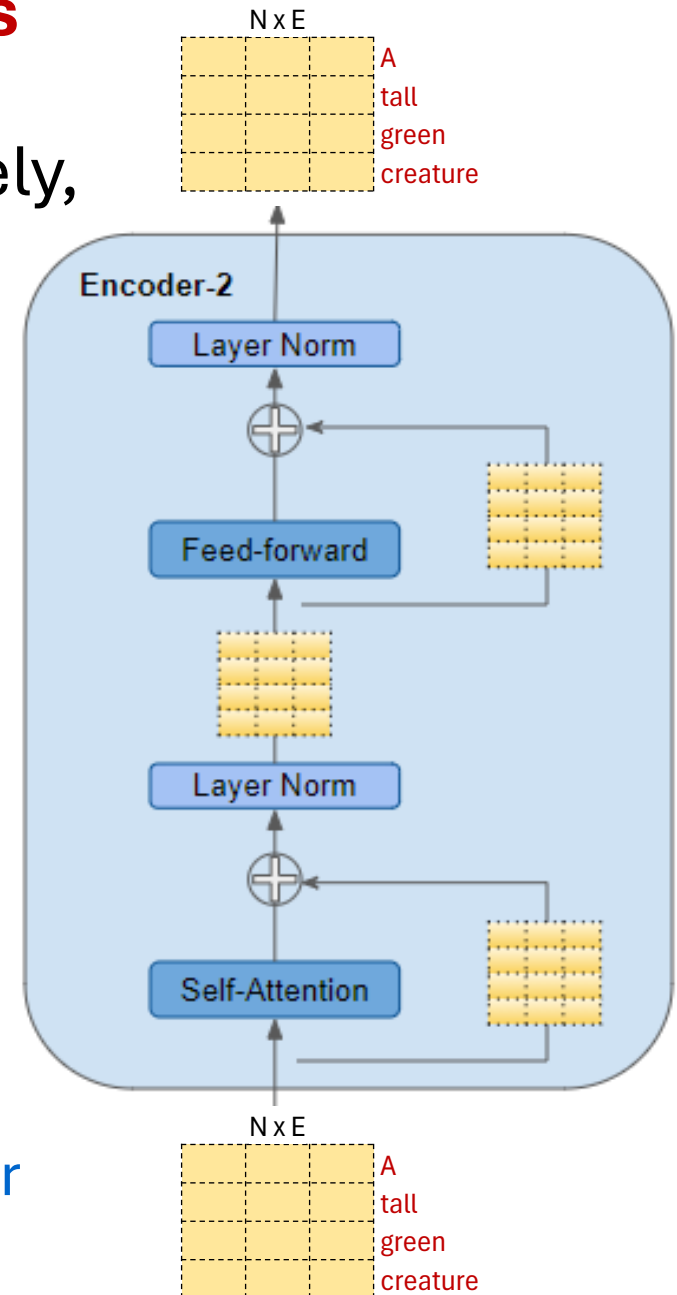
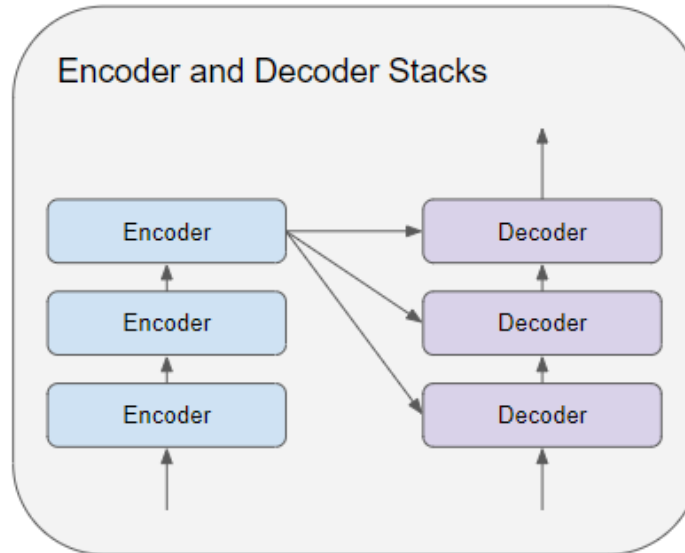
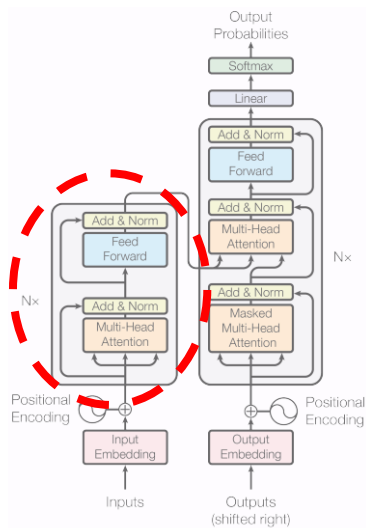
pos 40 = (... , 0, 1, .6, -.8, ...)

pos 60 = (... , 0, -1, -.6, -.8, ..)



AW: Encoder and Decoder Stacks

consist of several (usually six) Encoders and Decoders respectively, connected sequentially.



AW: Encoder

- First input is Embedding + Positional Encoding (Seq length $N \times$ Emb size E)
- Both the **Multi-head Self-attention layer** and **Feed-forward layer** have a residual skip-connection around them.

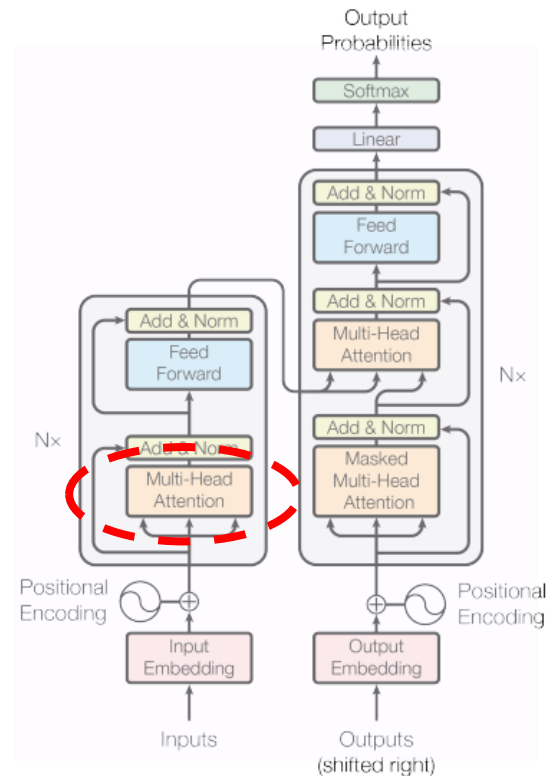
AW: Encoder Self-Attention

While processing a word, **Attention** enables the model to focus on other words as input that are closely related to that word.

Consider two sentences:

- The **cat** drank the milk because **it** was hungry.
- The cat drank the **milk** because **it** was sweet.

Different attentions will be given the word “it” in the above two sentences.



Encoder and Self-attention

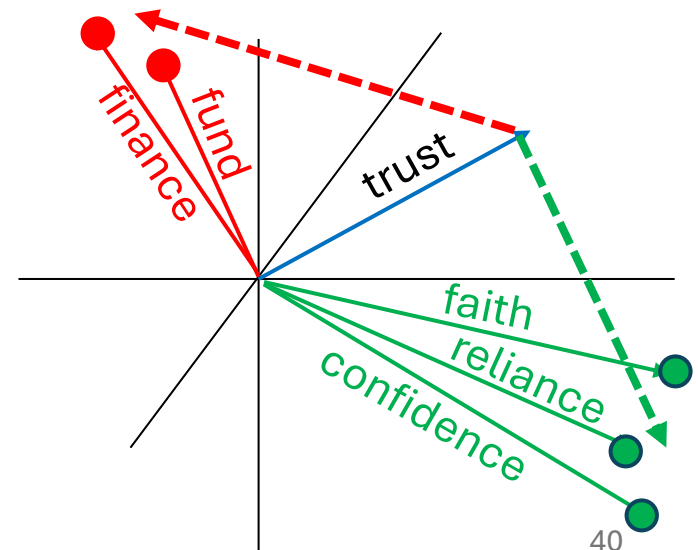
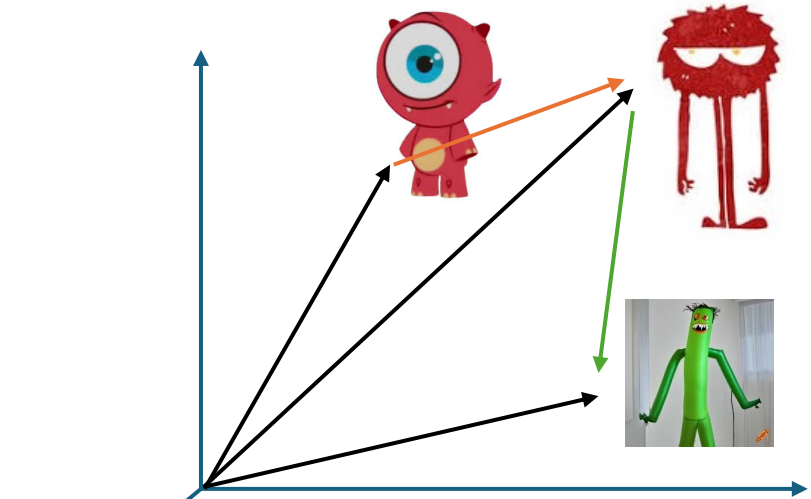
- Start with embedding for each word
- Compute another embedding for each word with attention

Embeddings for different contexts

He has no **trust** for his friends.

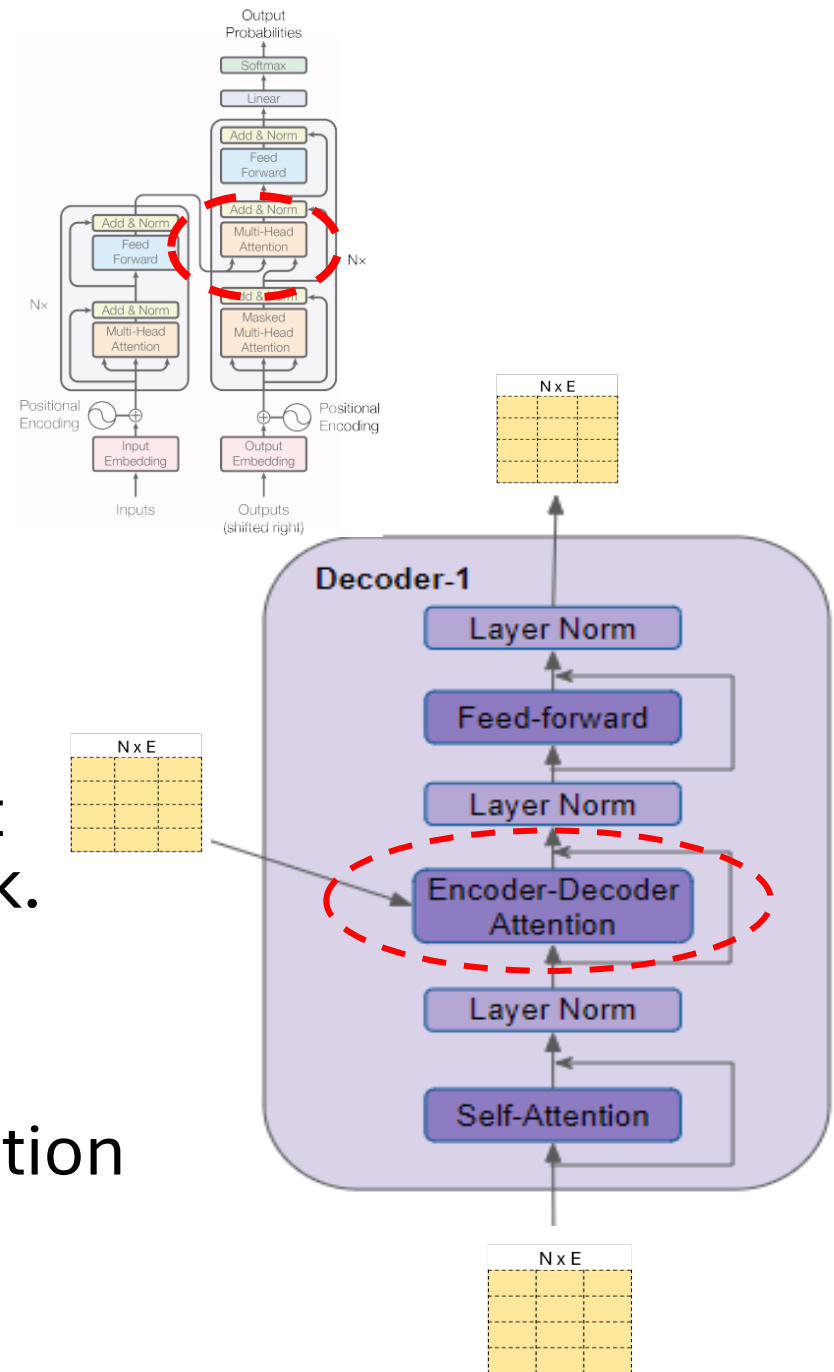
He has a **trust** fund.

A tall green **creature** lived
in the dark...



AW: Decoder

- Similar to encoder, except that Decoder has a second Multi-head attention layer, known as Encoder-Decoder attention layer.
- Encoder-Decoder attention layer combines two inputs:
 - self-attention layer below it
 - output of the Encoder stack.
- Encoder-Decoder-attention
 - target sequence pays attention to input sequence



Final output

De
1st Pos

nada
2nd Pos

END
3rd Pos

Select best words for
each position

Word Scores

(Seq Len x Vocab size)

Vocab size

1st Pos

2nd Pos

3rd Pos

One score for each
word in vocab, per
output position.

Word Scores

(Seq Len x Vocab size)

Vocab size

1st Pos

2nd Pos

3rd Pos

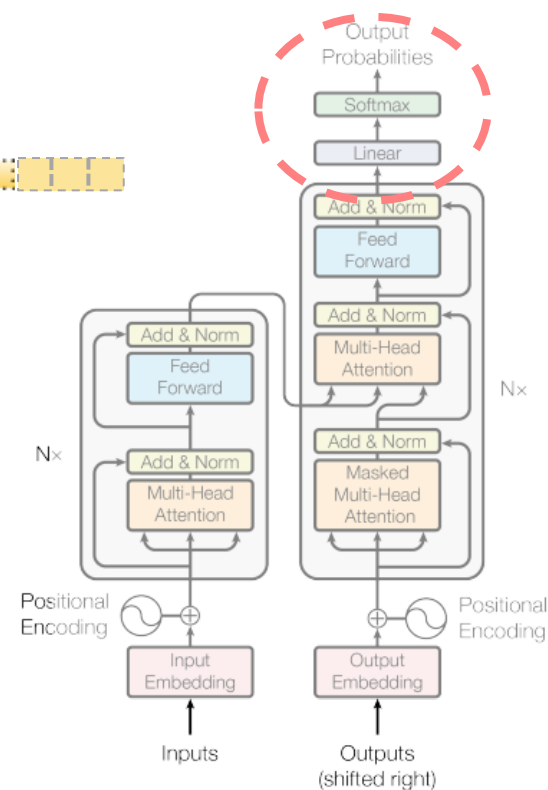
Linear

Emb

Seq

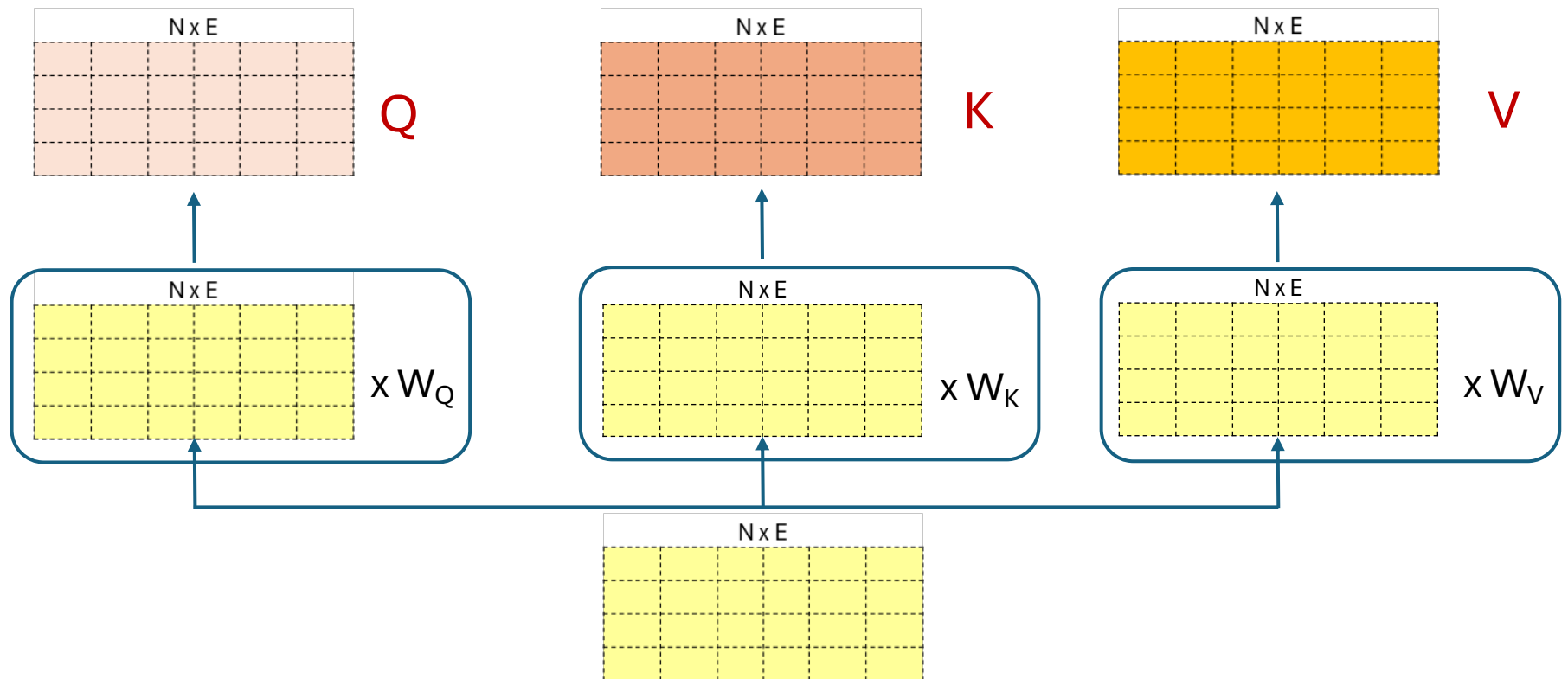
Decoder Stack

AW: Output Layer
(training)

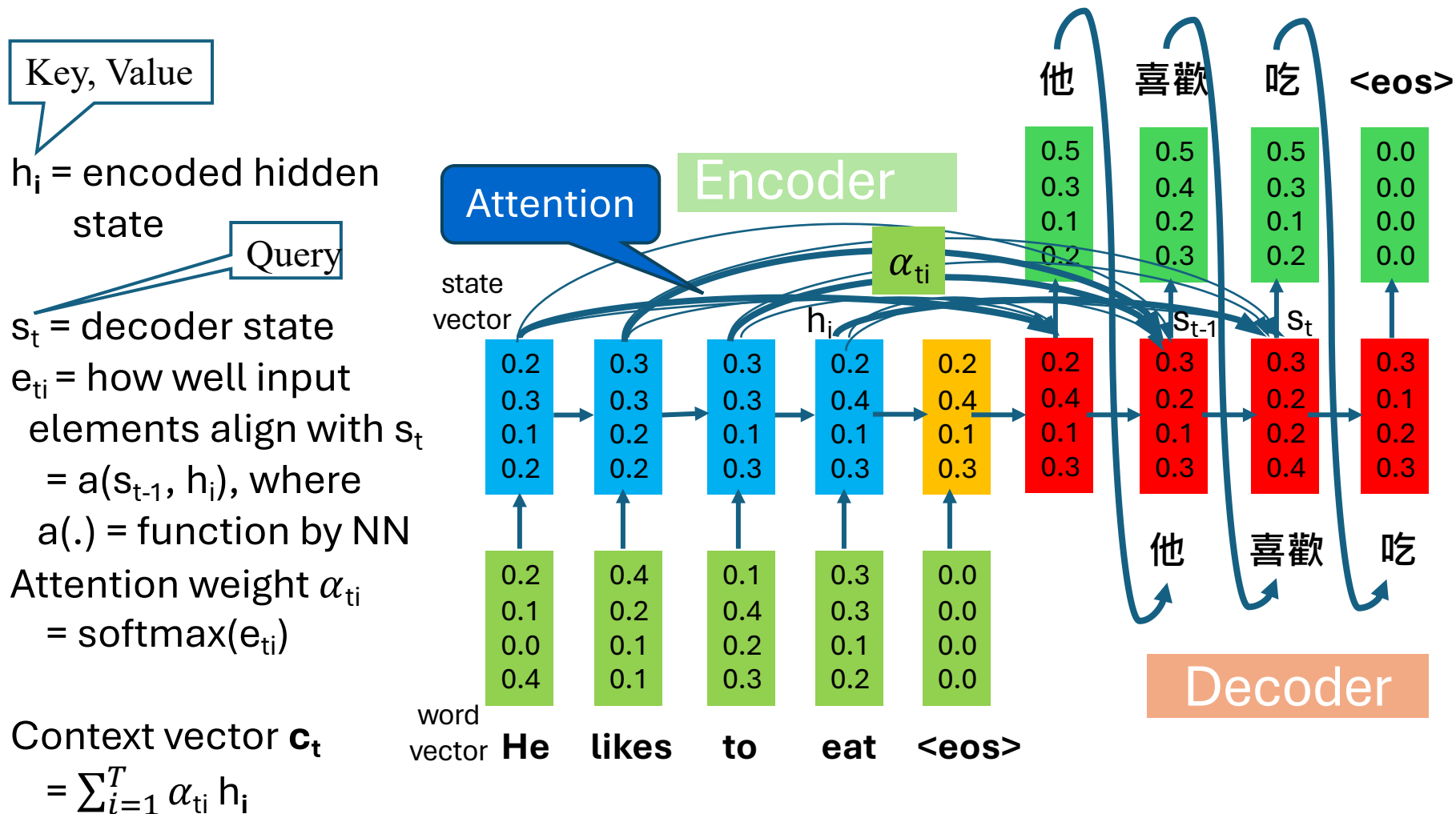


Step 1 for Attention layer: Create Q, K and V

- Create a **Q**uery, **K**ey and **V**alue for each token in sequence
- Three separate Linear layers for **Q**uery, **K**ey, and **V**alue.
- Each Linear layer has its own weights (to be learned) to produce **Q**uery, **K**ey, and **V**alue matrices



Going back to previous slide...



Attention (per head)

- Convert tokens (word) into vectors that represent the importance of the token in input sequence (per head).
- Each token is associated with **Q**uery, **K**ey and **V**alue where d is the head dimensionality of those

Key

h_i = encoded hidden

Query state

s_t = decoder state

e_{ti} = how well input elements align with s_t

$= a(s_{t-1}, h_i)$, where

$a(.)$ = function by NN

Attention Matrix

Attention weigh α_{ti}

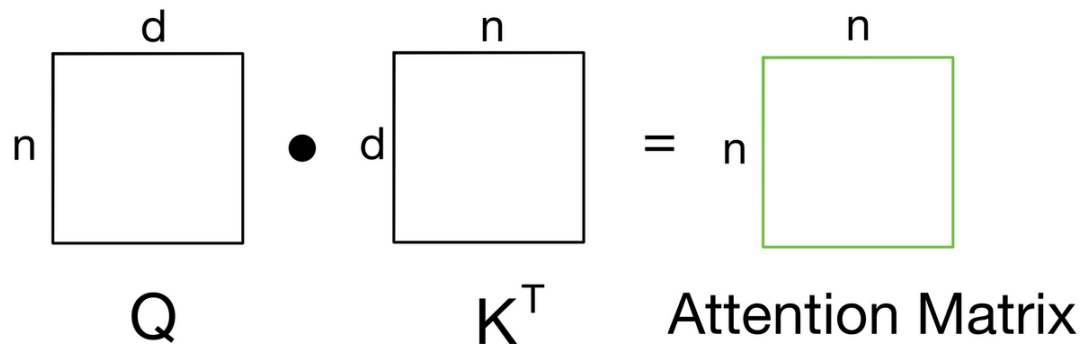
$= \text{softmax}(e_{ti})$

Context vector c_t

$= \sum_{i=1}^T \alpha_{ti} h_i$

Value

Consider a database with (**K**ey, **V**alue) pairs, attention is similar to retrieving information through matching **Q**uery with each **K**ey in the database (sequence).



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

A tall green **creature** lived in the dark...

A	tall	green	creature	lived	in	the	dark
E ₁	E ₂	E ₃	E ₄	E ₅	E ₆	E ₇	E ₈
×W _K	×W _K	×W _K	×W _K	×W _K	×W _K	×W _K	×W _K
K ₁	K ₂	K ₃	K ₄	K ₅	K ₆	K ₇	K ₈

$$A = E_1 \times W_Q = Q_1$$

$$\text{tall} = E_2 \times W_Q = Q_2$$

$$\text{green} = E_3 \times W_Q = Q_3$$

$$\text{creature} = E_4 \times W_Q = Q_4$$

$$\text{lived} = E_5 \times W_Q = Q_5$$

$$\text{in} = E_6 \times W_Q = Q_6$$

$$\text{the} = E_7 \times W_Q = Q_7$$

$$\text{dark} = E_8 \times W_Q = Q_8$$

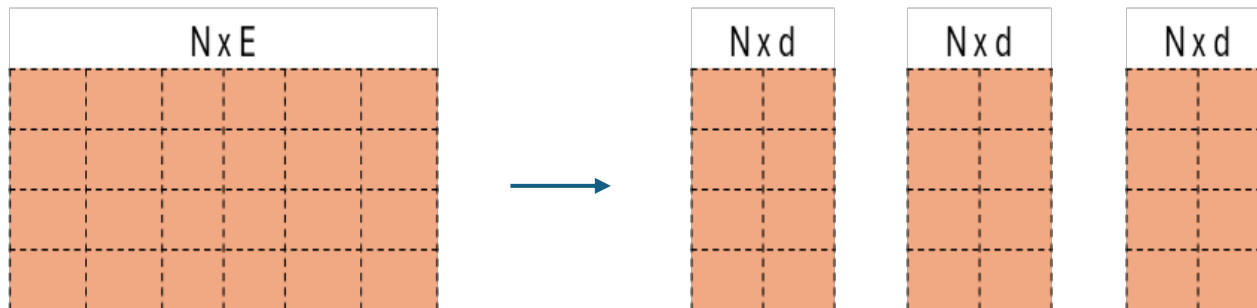
$$\text{softmax}\left(\frac{Q_j \times K_i^T}{\sqrt{\text{Query size}}}\right)$$

Multi-head Attention

- Multi-head self-attention mechanism allows the model to differentially weight parts of the sequence to infer meaning and context.
- The data gets split for multiple Attention heads so that each can process it independently.

$$\text{Query Size} = d = \frac{E}{\text{Number of heads}}$$

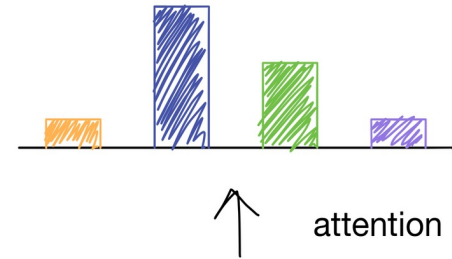
- Separate sections of Embedding can learn different meanings of each word in parallel to capture different types of relationship (e.g., gender, cardinality, adjective, action ...)
- Separate Q, K and V.



Step 2: Calculate similarity between Q and K

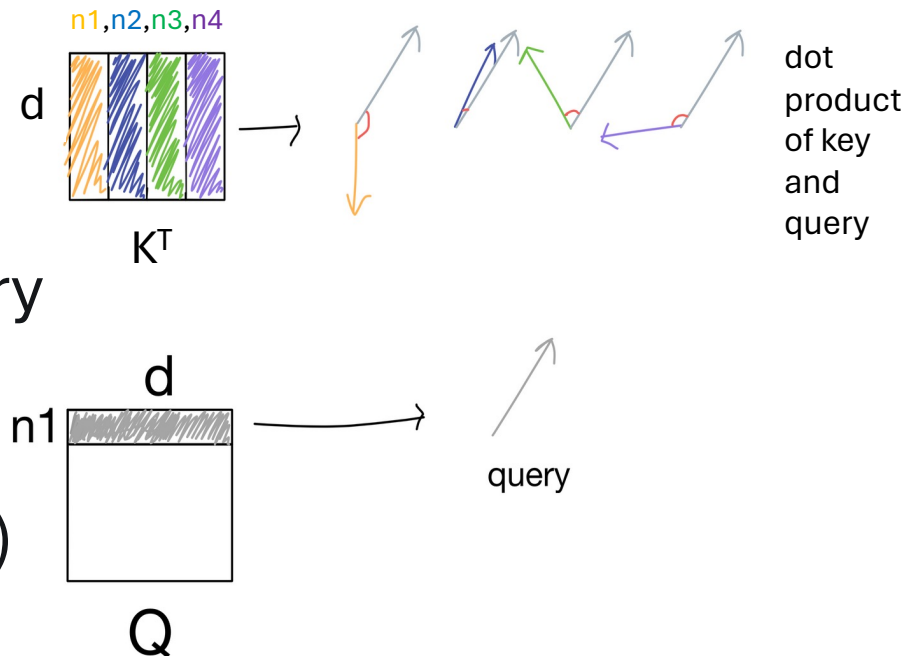
- Calculates similarity between **Q**uery vector (step1) and **K**ey vector of every other token by dot product.

$$\begin{array}{|c|} \hline Q \\ \hline N \times d \\ \hline \end{array} \times \begin{array}{|c|} \hline K^T \\ \hline d \times N \\ \hline \end{array} = \begin{array}{|c|} \hline \text{Attention} \\ \text{Matrix} \\ \hline N \times N \\ \hline \end{array}$$



Each **Q**uery of a token can be considered as a row vector in the input matrix.

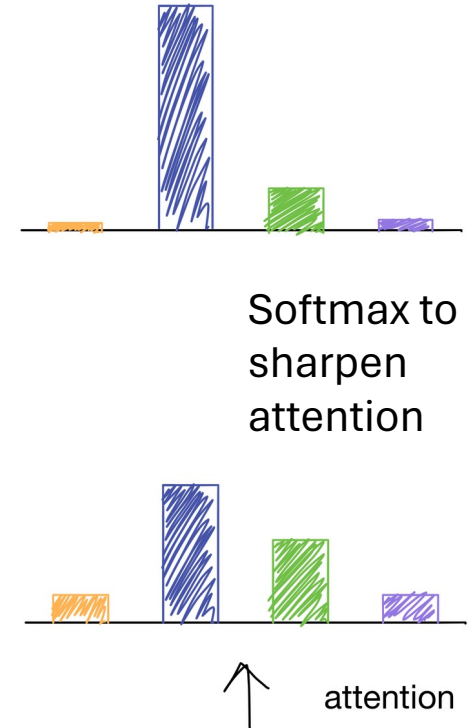
Similarity between each **Q**uery vector with each **K**ey of a token
(also represented by a vector)
is measured by dot product.

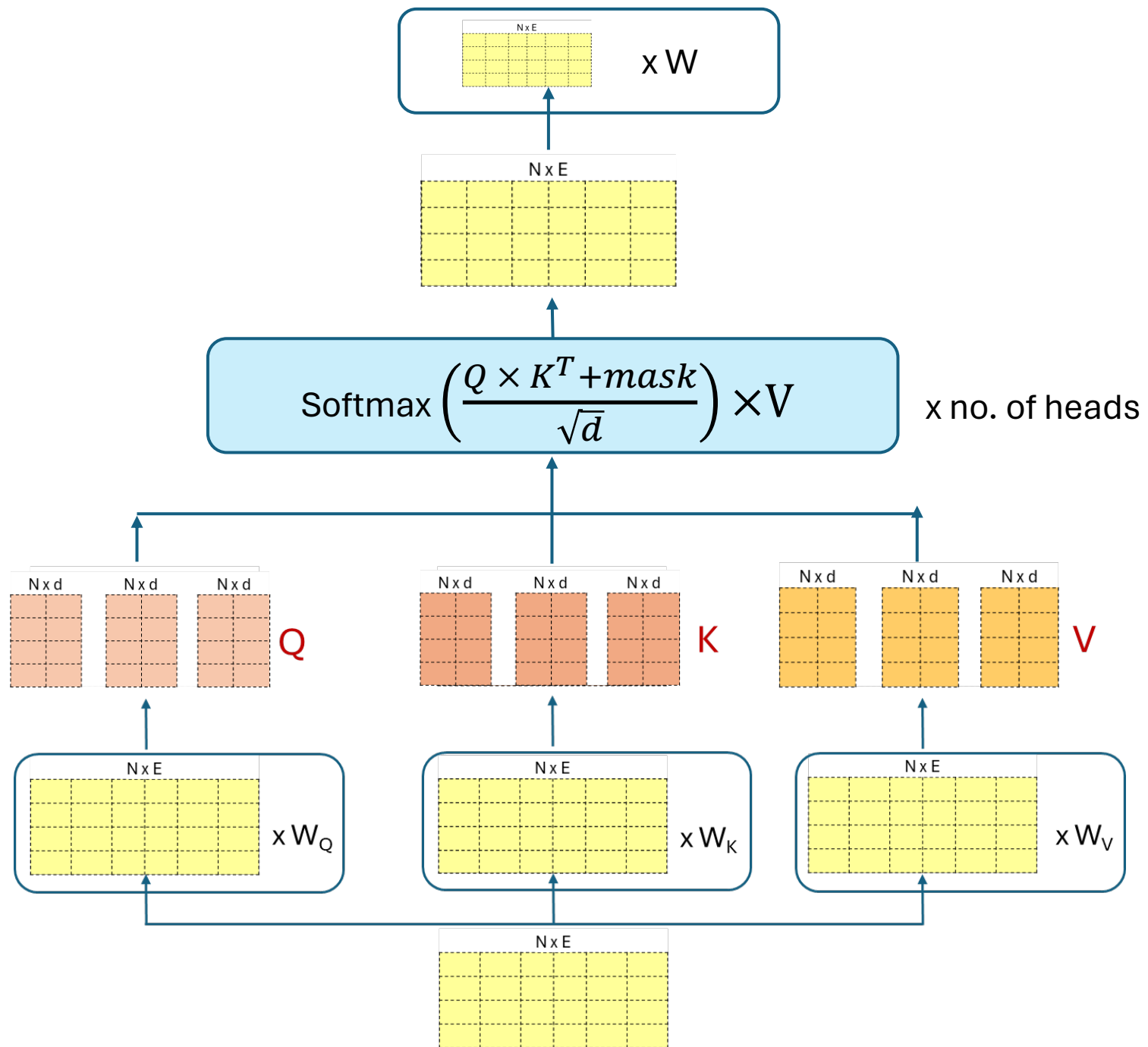


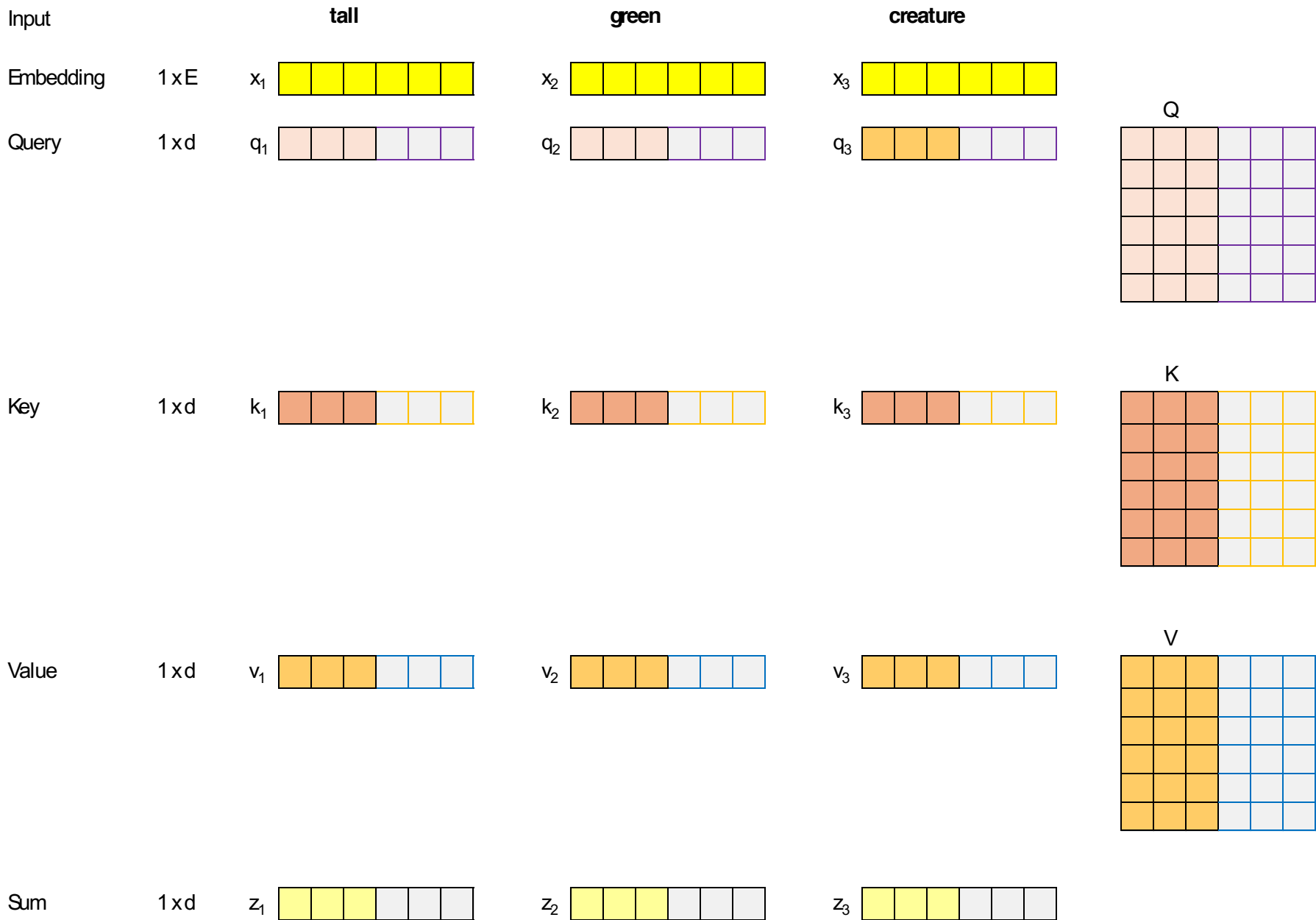
Steps 3&4: normalize and generate vector

- Step 3: Generates normalized weights (step 2) through softmax
- Step 4: Generate a vector representing the importance of the token within the sequence by multiplying the weights (step3) by the value vectors of each token.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



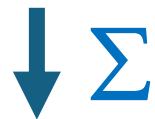




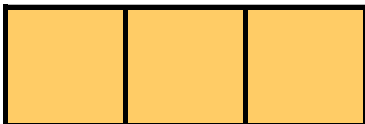
One Attention matrix per head

	tall	green	creature
tall			
green			
creature	$\left[\text{softmax} \left(\frac{q_3 \cdot k_1}{\sqrt{d}} \right) \right] v_1$	$\left[\text{softmax} \left(\frac{q_3 \cdot k_2}{\sqrt{d}} \right) \right] v_2$	$\left[\text{softmax} \left(\frac{q_3 \cdot k_3}{\sqrt{d}} \right) \right] v_3$

$\left[\text{softmax} \left(\frac{q_3 \cdot k_1}{\sqrt{d}} \right) \right]$ is a number



v_i

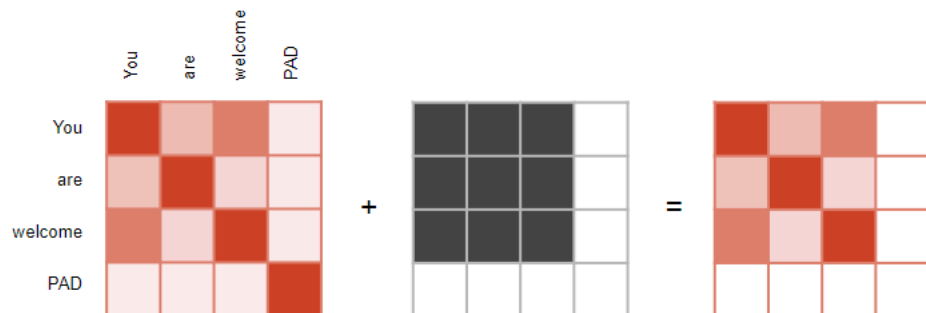


z_3

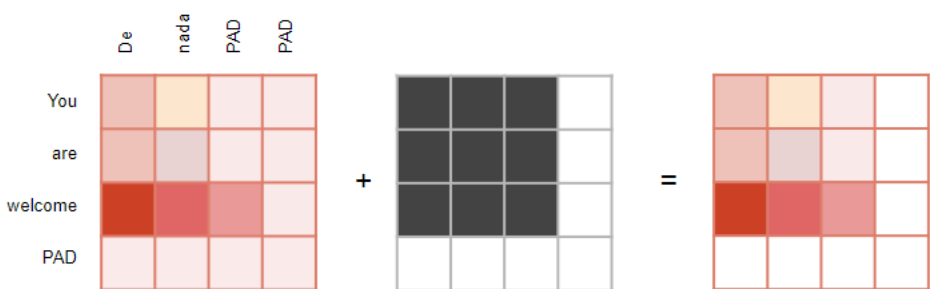


Attention masks

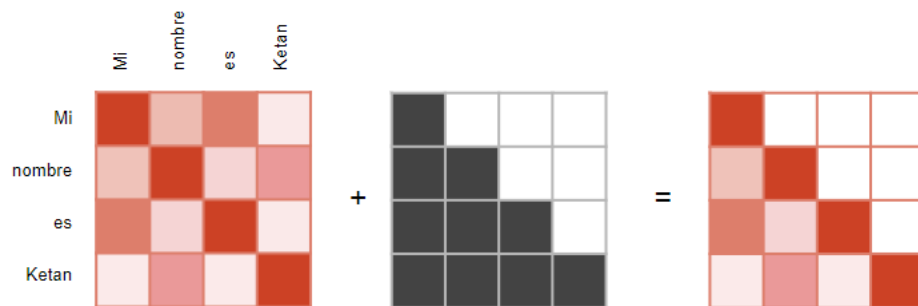
- The masked-out elements (white squares) are set to $-\infty$, so that Softmax turns those values to 0.
- **Encoder Self-attention and Encoder-Decoder-attention** -- masking zero attention to padding (different input lengths).
- **Decoder Self-attention** masking serves to prevent decoder from 'peeking' ahead at the rest of the target sentence when predicting the next word.



Encoder Self-Attention Scores



Encoder-Decoder Attention Scores



Decoder Self-Attention Scores

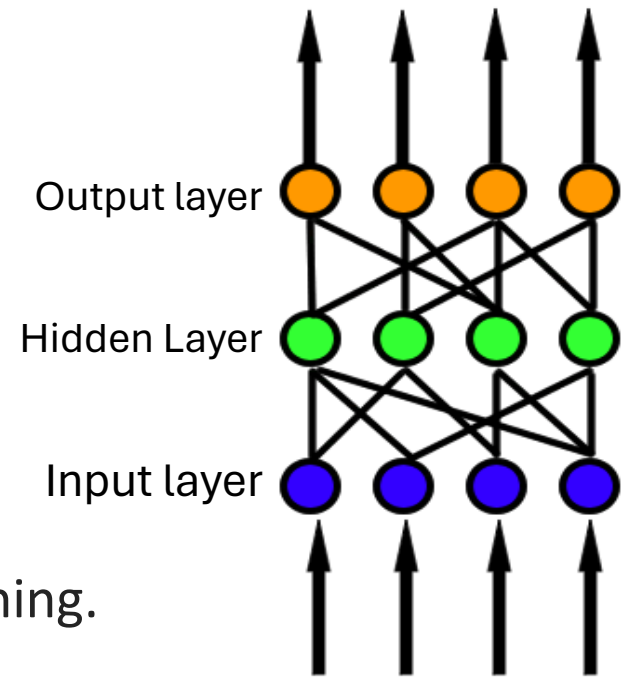
Feedforward

In a feedforward network, information only moves forward.

1. **Linear Transformation**
fully connected layer, or dense layer
2. **Activation Function** –
introduces non-linearity
3. **Dropout** - prevents overfitting
4. **Layer Normalization**
- stabilizing network and speeding up training.

Why is Feedforward Important?

- a) Learn complex patterns and relationships in data
- b) Versatility - can be used in a variety of tasks
- c) Compatibility with Other Operations – combine their strengths
- d) Scalability and performance – independent and parallelized
- e) Simplicity + efficiency - matrix multiplications + non-linear activation
- f) Enabling Deeper Networks - batch normalization and residual connections



Summary on Transformer

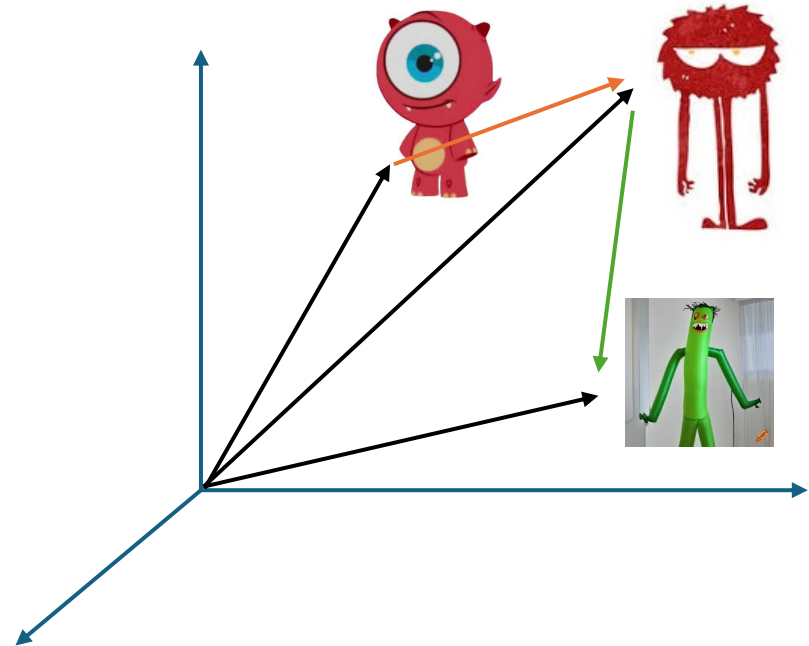
Encoder

- Start with embedding for each word
- Use embedding to generate the **key**, **query** and **value** vectors for each word
- Compute another embedding for each word with attention

Decoder

- Generate the output sentence word by word while considering the representation vector that was created by the encoder and the decoder's last output.

A tall green **creature** lived in the dark...



Why Transformer Performs so well?

- **LSTM** is widely used before 2017, it cannot capture long-term dependency well, not more than 7-15 tokens.
- **Google's Transformer** (NN architecture with attention mechanism) designed for seq2seq tasks (e.g., translation), has many benefits over conventional sequential models (e.g., LSTM, RNN, etc),

1. **Variable Size of Embedding**
depends on the the input size
(instead of RNN fixed size state vector)
without losing information
2. **Direct Interaction** between tokens
(RNN or LSTM sequential processing
depends on distance between tokens)
3. **Parallelizability**
for all pairs of token interactions

